Stage 16N Inhomogeneous Plate-with-Hole Cycle-Jump Benchmark
Living Technical Report

Master Thesis Preparation

Last updated: 2026-06-16

Abstract

This living report records the geometry, numerical setup, HPC configuration, reference results, reinjection tests, diagnostic failures, and current conclusions for Stage 16N. The stage uses an inhomogeneous Abaqus plate-with-hole benchmark with a NEML-equivalent Chaboche UMAT. The full 1000-cycle non-jump reference has been completed and is used as the validation baseline for later fixed and adaptive cycle-jump simulations. The current fixed-jump conclusion is frozen at a narrow accepted cycle-100 jump, `B1D5/B1D5_EQ`: 100 $\rightarrow$ 105 $\rightarrow$ 250 with 2.34986% maximum primary scalar-metric error. Larger and later manually initialized jumps remain blocked because `SDVINI/SIGINI`-based reinjection does not robustly reconstruct the later-cycle inhomogeneous finite-element state.

Contents

1 Purpose and Current Status

The purpose of Stage 16N is to move from single-element or homogeneous cycle-jump demonstrations to an inhomogeneous finite-element benchmark. The model is deliberately more demanding: a plate with a central hole produces localized cyclic plasticity near the hole, so the method must be judged not only from global reaction-force loops but also from local stress and state-variable evolution.

Table 1: Current Stage 16N infrastructure and validation status.

Item	Status
1000-cycle full non-jump reference	Completed
16-CPU production policy	Locked
Abaqus threaded launcher	Verified
Adaptive ΔN table	Completed
Exact reinjection scaffold	Prepared
HPC mail policy	Fixed
B0-1 exact reinjection solver run	Completed
B0-1 scientific result	Practical baseline accepted
B0 initialization-only audit	Completed
B0-2 / B0-3 exact reinjection jobs	On hold
Fixed state-initialized jump validation	Scaffold prepared
Adaptive cycle-jump workflow	On hold

2 Geometry and Model Definition

The Stage 16N model is a three-dimensional plate-with-hole benchmark. It is intended to create localized cyclic evolution near the hole while retaining a global hysteresis response that can be compared across full-reference, exact-reinjection, fixed-jump, and adaptive-jump runs.

Table 2: Stage 16N model geometry and mesh summary.

Quantity	Value
Plate length	20.0
Plate height	10.0
Plate thickness	1.0
Central hole radius	1.0
Mesh spacing	$\Delta x = \Delta y = 0.25$
Element type	C3D8
Nodes	6642
Elements	3148
Hole-ring elements	60
Material model	NEML-equivalent Chaboche UMAT
State variables	27 SDVs

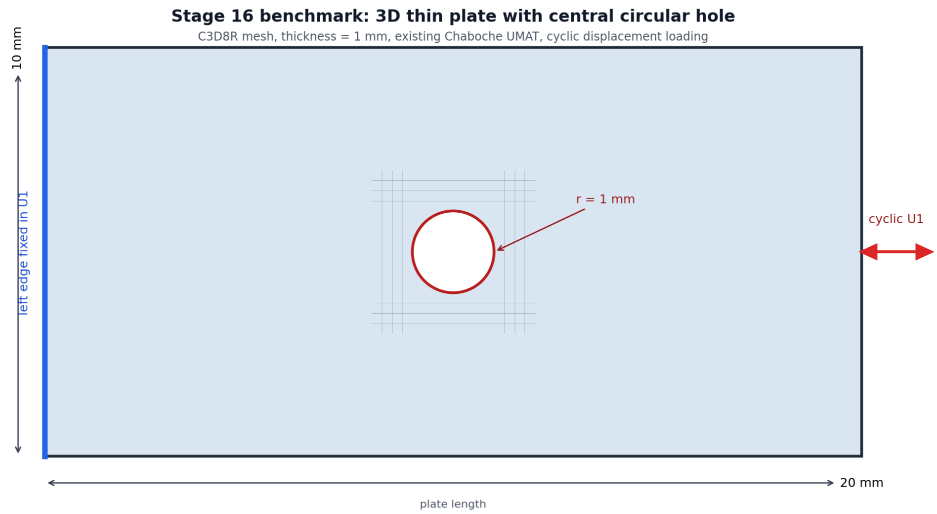

Figure 1: Plate-with-hole geometry used for the Stage 16N benchmark.

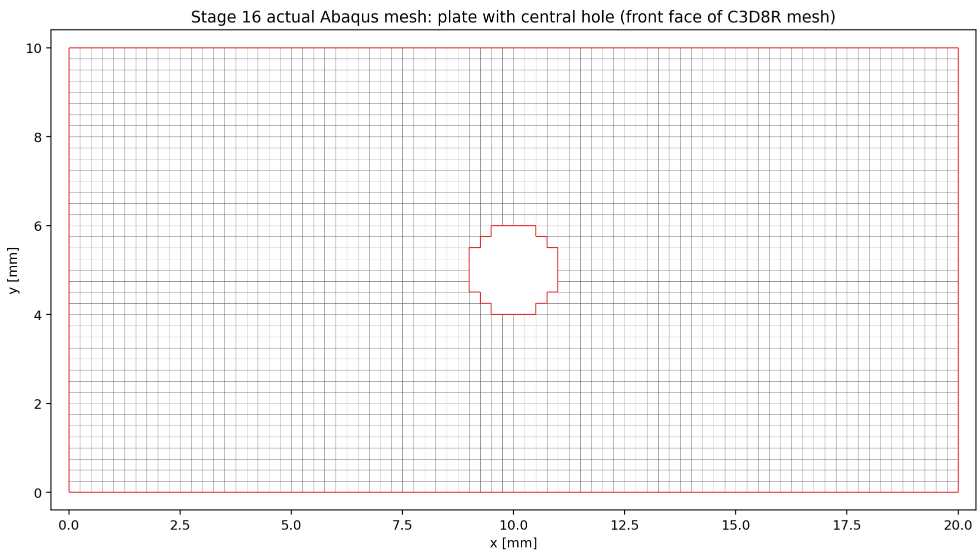

Figure 2: Actual structured C3D8 mesh with the element-wise approximation of the central hole.

2.1 Boundary Conditions and Loading

The left edge is fixed in the axial direction. Two anchor nodes constrain rigid-body modes. The right edge receives a cyclic axial displacement amplitude of ± 0.10. The full reference contains 1000 full non-jump cycles. Selected field output is stored at cycles 1, 2, 10, 50, 100, 250, 500, 750, and 1000.

3 HPC and Production Policy

The first corrected 1000-cycle parallel reference used 30 OpenMP threads, but PBS accounting showed that the average effective CPU usage was far below the requested 30 cores. A 16-CPU verification run was therefore used to lock a more resource-efficient production policy.

Table 3: Locked Stage 16N production setting.

Setting	Value
Production default	1 MPI rank × 16 OpenMP threads
PBS request	`select=1:ncpus=16:mpiprocs=1:ompthreads=16`
Abaqus launch	`cpus=16 mp_mode=threads`
Expected message-file line	`1 MPI RANK x 16 THREADS`
Maximum walltime per job	24:00:00
Maximum simultaneous jobs	2
Maximum active Stage 16N cores	32

This policy is not claimed to perfectly saturate all 16 CPUs. It is a resource-efficient compromise that prevents the earlier serial fallback and avoids reserving 30 CPUs for a workload that did not use them efficiently.

3.1 Two-Job Scheduling Policy

From 2026-06-06 onward, Stage 16N solver planning assumes at most two simultaneous production jobs. Each job requests 16 CPU cores with one MPI rank and 16 OpenMP threads, so the intended active usage is 32 CPU cores. The two slots are used only after the relevant method gate has passed. First-time method checks, such as the first fixed cycle-jump case B1, run alone until extracted and compared. If B1 passes, B2 and B3 can be submitted together. If B1 fails badly, B2 and B3 remain held and the two slots are used instead for smaller or diagnostic B1 variants.

3.2 Mail Notification Policy

Future PBS jobs must explicitly request mail on abort, begin, and end:

```
#PBS -m abe
#PBS -M pr21vyci@mailserver.tu-freiberg.de
```

This was added because PBS accounting previously showed `Mail_Points = a`, which only requests abort/failure mail.

4 Full 1000-Cycle Non-Jump Reference

The full non-jump reference was completed using Abaqus/Standard with the corrected threaded launcher.

Table 4: Completed 1000-cycle reference run.

Quantity	Value
PBS job	1335408.mmaster02
Abaqus job	`stage16n_parallel_max_reference_1000cycles`
Walltime	17:56:43
CPU time	178:03:42
Requested resources	30 CPUs, 135 GB
Abaqus parallelism	1 MPI rank × 30 threads
Completed cycles	1000
Last partial cycle	none

Table 5: Reference evolution from cycle 1 to cycle 1000.

Quantity	Cycle 1	Cycle 1000	Change
RF1_max	2356.834	2908.244	+23.40%
RF1_min magnitude	2509.712	3074.823	+22.52%
loop_area_abs	433.822	577.068	+33.02%
HOLE_RING_MISES_MAX	398.503	492.827	+23.67%
HOLE_RING_SDV1_MAX	0.0561	26.0519	–
HOLE_RING_SDV8_MAX	0.2996	92.4455	–
HOLE_RING_SDV11_MAX	7.9019	61.3076	–

The evolution clearly exceeds the original continuation thresholds for global loop quantities and local hole-ring state variables. This confirms that the plate-with-hole benchmark is meaningful for cycle-jump validation.

5 Adaptive ΔN Table

The adaptive ΔN table was computed from the completed 1000-cycle reference. It uses dense global cycle metrics and selected local field-output anchors. The controlling variable is the most sensitive monitored quantity, not only the global reaction force.

Table 6: Conservative adaptive ΔN estimates from the 1000-cycle reference.

Base cycle	Next anchor	Target	ΔN	Controlling variable
1	2	2	1	HOLE_RING_SDV8_MAX
2	10	3	1	HOLE_RING_SDV8_MAX
10	50	14	4	HOLE_RING_SDV8_MAX
50	100	61	11	HOLE_RING_SDV1_MAX
100	250	124	24	HOLE_RING_SDV1_MAX
250	500	299	49	HOLE_RING_SDV1_MAX
500	750	575	75	HOLE_RING_SDV1_MAX
750	1000	849	99	HOLE_RING_MISES_MAX

The table shows that local state variables near the hole strongly restrict the allowable jump size. This supports the decision that any later fixed/adaptive jump validation must include local metrics, not only global RF loops.

5.1 Reinjection-Aware Controller Update

After the B0 initialization audit, HOLE_RING_SDV8_MAX was downgraded from a hard pass/fail controller to a diagnostic-only variable because it already shows a large mismatch immediately after manual SDVINI/SIGINI initialization. A reinjection-aware table was therefore generated with SDV8 retained in diagnostic output but removed from the hard controller list.

Table 7: Reinjection-aware adaptive ΔN estimates with `SDV8` diagnostic-only.

Base	Next anchor	Target	ΔN	Controller
1	2	2	1	HOLE_RING_SDV11_MAX
2	10	3	1	HOLE_RING_SDV1_MAX
10	50	15	5	HOLE_RING_SDV1_MAX
50	100	61	11	HOLE_RING_SDV1_MAX
100	250	124	24	HOLE_RING_SDV1_MAX
250	500	299	49	HOLE_RING_SDV1_MAX
500	750	575	75	HOLE_RING_SDV1_MAX
750	1000	849	99	HOLE_RING_MISES_MAX

The first conservative fixed-jump gate remains cycle 100 to cycle 125, followed by normal continuation to cycle 250. This row is still controlled by `HOLE_RING_SDV1_MAX`, not by diagnostic-only `SDV8`.

6 Stage 16N-B: State-Initialized Continuation Verification

Before any real cycle jump is allowed, the workflow must prove that stress and state-variable fields can be transferred into a new Abaqus run. The route used here is manual state-initialized continuation through `SDVINI/SIGINI`; it is not a native Abaqus restart. The originally intended verification cases were:

- B0-1: exact cycle 100 state, continue normally to cycle 250.
- B0-2: exact cycle 250 state, continue normally to cycle 500.
- B0-3: exact cycle 500 state, continue normally to cycle 1000.

B0-2 and B0-3 remain on hold. The B0-1 and initialization-audit results are now treated as the measured reinjection baseline rather than as proof of an exact local restart.

6.1 State Reader Implementation

The initial CSV preload design failed under threaded Abaqus because multiple threads attempted to load the same file concurrently. The state-transfer implementation was therefore changed to a direct-access binary reader. Each record contains:

```
S1, S2, S3, S4, S5, S6, SDV1, ..., SDV27
```

The record number is:

```
record = (NOEL - 1) * 8 + NPT
```

On the Abaqus/Intel Fortran environment used here, direct-access `RECL` is interpreted in 4-byte words. Therefore the correct value for 33 double-precision values is:

```
RECL = 66
```

7 B0-1 Cycle-100 to Cycle-250 State-Initialized Continuation

B0-1 completed the Abaqus solver successfully. PBS reported exit status 2 only because the original postprocessing path failed after solver completion; extraction was rerun manually afterward.

Table 8: B0-1 run accounting and solver status.

Quantity	Value
PBS job	1336497.mmaster02
Route	cycle 100 state → cycle 250
PBS walltime	01:41:36
PBS CPU time	10:52:42
Requested CPUs	16
Abaqus solver	completed
Abaqus parallelism	1 MPI rank × 16 threads
Scientific status	Practical baseline accepted

Table 9: B0-1 comparison against the 1000-cycle reference at cycle 250.

Quantity	Relative error
`RF1_max`	0.259048%
`RF1_min`	0.687538%
`loop_area_abs`	0.967300%
`HOLE_RING_MISES_MAX`	7.63648%
`HOLE_RING_S11_MAX_ABS`	9.58753%
`HOLE_RING_SDV1_MAX`	0.360172%
`HOLE_RING_SDV8_MAX`	10.9164%
`HOLE_RING_SDV11_MAX`	7.27213%

The global response is excellent, but the local hole-ring stress and state-variable errors are too large for a clean exact-restart claim.

8 B0-1 Local Diagnostic Review

A pointwise hole-ring diagnostic was run to determine whether the high local error was only a maximum-location artifact. It was not. The reference and reinjection maxima for `SDV8` occurred at the same element and integration point.

Table 10: B0-1 SDV8 argmax diagnostic at cycle 250.

Quantity	Value
Reference SDV8 argmax	element 1242, IP 7
Reinjection SDV8 argmax	element 1242, IP 7
Same argmax location	true
Reference value	91.1466598511
Reinjection value	81.1967315674
Argmax relative error	10.9164%

This made B0-1 a real local mismatch rather than a harmless maximum-location shift.

9 B0 Cycle-100 Initialization-Only Audit

The initialization audit was created to answer whether the local mismatch already exists immediately after exact cycle-100 initialization, before any cycle-100 to cycle-250 continuation.

Table 11: Cycle-100 binary reader and initialization audit.

Quantity	Value
CSV-vs-binary maximum absolute error	3.55×10^{-15}
Common hole-ring element/IP records	480
`SDV8` median relative error	12.7022%
`SDV8` 95th percentile relative error	150.665%
`SDV8` maximum relative error	479.090%
`SDV8` argmax location	same element/IP, 1242/7

The binary extraction and reader pipeline is correct to machine precision. Therefore the local mismatch is not caused by the Python binary writer or direct-access file format. The mismatch appears during manual Abaqus initialization and the tiny equilibrium step.

9.1 Initialization Audit Interpretation

The most likely interpretation is that `SDVINI/SIGINI` inject stress and state variables correctly, but the manual state-initialized model does not reconstruct the compatible displacement, strain, solver-history, and full equilibrium state of the original reference analysis. Strain-like local state variables such as `SDV8` are therefore adjusted during the first equilibrium solve. This means B0-1 is useful as a practical state-initialized continuation test, but it is not a mathematically exact Abaqus restart for local fields.

10 Reinjection Error Floor and Revised Metric Policy

The B0 audit closes Stage 16N-B with the following conclusion:

> Global continuation works, binary state transfer works, but local exact restart is not achieved with `SDVINI/SIGINI` alone.

Future fixed and adaptive jump results must separate the manual reinjection error floor from the additional error caused by cycle-jump extrapolation. The primary pass/fail metrics are `RF1` maximum/minimum, loop area, global hysteresis shape, `HOLE_RING_SDV1_MAX`, `HOLE_RING_SDV11_MAX`, and `HOLE_RING_MISES_MAX`. `HOLE_RING_S11_MAX_ABS` remains important, but it is now treated as a diagnostic warning metric rather than a sole hard rejection variable, because the B1D3/B1D4/B1D5 boundary study showed non-monotonic local stress sensitivity. `HOLE_RING_SDV8_MAX` also remains diagnostic-only because it is already affected by initialization before any cycle-jump approximation is applied.

11 Stage 16N-C Fixed State-Initialized Jump Scaffold

Stage 16N-C now begins with one conservative fixed jump:

Table 12: First fixed state-initialized jump case.

Quantity	Value
Case name	`B1_100_to_125_to_250`
Base state	cycle 100
Interpreted jump target	cycle 125
Normal Abaqus continuation	cycles 126–250
Comparison endpoint	cycle 250
Initial state strategy	zero-order hold from cycle 100
Baseline for interpretation	B0-1 cycle 100 to 250 continuation

The zero-order state strategy is intentionally conservative. It measures the error added by skipping cycles 101–125 on top of the known manual reinjection floor. If this case gives acceptable global error and controlled primary local errors, the next fixed jump cases can be expanded to cycle 250 to 300, cycle 500 to 575, and cycle 750 to 850.

11.1 Prepared-Only B2 and B3 Cases

B2 and B3 may be prepared while B1 is running, but their solver jobs are gated by the B1 result. They must not be submitted until B1 has been extracted, compared against the 1000-cycle reference, and interpreted against the B0 reinjection baseline.

Table 13: Prepared-only fixed-jump cases held behind the B1 gate.

Case	Base state	Jump target	Compare endpoint
`B2_250_to_300_to_500`	cycle 250	cycle 300	cycle 500
`B3_500_to_575_to_750`	cycle 500	cycle 575	cycle 750

If B1 passes, B2 and B3 are the first paired submission under the new two-job policy. If B1 fails, the paired slots should instead be used for B1-small and B1-diagnostic variants.

11.2 B1 Result

B1 completed successfully as an Abaqus job, but it did not pass the fixed-jump method gate. The comparison at cycle 250 gave good global errors but unacceptable primary local state errors.

Table 14: B1 fixed-jump comparison against the 1000-cycle reference and B0 baseline.

Quantity	B1 total error	B0 baseline	Additional error
`RF1_max`	0.361433%	0.259048%	0.102384%
`RF1_min`	2.27018%	0.687538%	1.58264%
`loop_area_abs`	1.26223%	0.967300%	0.294934%
`HOLE_RING_MISES_MAX`	2.40141%	7.63648%	-5.23507%
`HOLE_RING_S11_MAX_ABS`	1.21869%	9.58753%	-8.36884%
`HOLE_RING_SDV1_MAX`	10.2916%	0.360172%	9.93139%
`HOLE_RING_SDV8_MAX`	16.1769%	10.9164%	5.26047%
`HOLE_RING_SDV11_MAX`	11.2777%	7.27213%	4.00560%

The B1 status is `review`, controlled by `HOLE_RING_SDV11_MAX`. Therefore B2 and B3 remain held. The two-job policy should next be used for smaller or diagnostic B1 variants rather than larger fixed jumps.

11.3 B1 Jump-Size Sensitivity

The next two-job batches reduced the first fixed jump from cycle 100 to smaller target cycles while keeping the same comparison endpoint at cycle 250. This directly tests whether the local hole-ring error decreases when fewer cycles are skipped.

Table 15: B1 fixed-jump sensitivity at cycle 250.

Case	Jump target	Skipped cycles	Status	Controlling quantity
`B1Q_100_to_106_to_250`	106	6	review, 5.75372%	`HOLE_RING_MISES_MAX`
`B1S_100_to_112_to_250`	112	12	review, 9.34203%	`HOLE_RING_S11_MAX_ABS`
`B1_100_to_125_to_250`	125	25	review, 11.2777%	`HOLE_RING_SDV11_MAX`

The sensitivity study confirms that the original jump from cycle 100 to 125 was too aggressive for local hole-ring variables. Reducing the target to cycle 106 reduced the maximum primary error to approximately 5.75%, but this remained above the strict 5% gate. The useful scientific conclusion is that the adaptive jump-size estimate near cycle 100 was too optimistic and must be reduced by an early-cycle safety factor. Since the reinjection-aware table suggested approximately $\Delta N \approx 24$ at cycle 100, the practical early safety factor is approximately

$$\frac{5}{24} \approx 0.21.$$

This supports an early-cycle safety-factor range of approximately 0.20–0.25, controlled by local field quantities rather than by `RF1` alone.

11.4 B1D3/B1D4/B1D5 Boundary Check

A stricter boundary check was then performed with target cycles 103, 104, and 105. The result is useful, but non-monotonic:

Table 16: B1D3/B1D4/B1D5 boundary check at cycle 250.

Case	Jump target	Skipped cycles	Status	Controlling quantity
`B1D3_100_to_103_to_250`	103	3	review, 9.13095%	`HOLE_RING_S11_MAX_ABS`
`B1D4_100_to_104_to_250`	104	4	review, 9.34307%	`HOLE_RING_S11_MAX_ABS`
`B1D5_100_to_105_to_250`	105	5	pass, 2.34986%	`HOLE_RING_SDV1_MAX`
`B1Q_100_to_106_to_250`	106	6	review, 5.75372%	`HOLE_RING_MISES_MAX`

B1D5 remains a useful successful case:

`100 -> 105 -> 250`, with maximum primary error 2.34986%.

However, B1D3 and B1D4 both failed under the local stress criterion even though they used shorter jumps. B1D3 was controlled by `HOLE_RING_S11_MAX_ABS` at 9.13095%, and B1D4 was controlled by the same metric at 9.34307%. Therefore the cycle-100 boundary is not robust under a hard local-stress maximum criterion. The isolated B1D5 pass should not be generalized as a universal accepted early-cycle rule.

11.5 B1D4 Pointwise Diagnostic

B1D4 was diagnosed at the hole-ring element/integration-point level using the 1000-cycle reference ODB and the B1D4 ODB at cycle 250. The diagnostic compared 480 common hole-ring element/IP records and checked both pointwise errors and argmax locations.

Table 17: B1D4 pointwise diagnostic summary at cycle 250.

Variable	Mean error	Median error	95th percentile	Max pointwise error
S11_ABS	13.8050%	11.6186%	35.2953%	121.4232%
MISES	8.05921%	–	–	–
SDV1	1.64737%	–	–	–
SDV8	131.337%	–	–	–
SDV11	50.4912%	–	–	–

For the controlling stress metric, the reference and B1D4 argmax location were the same:

S11_ABS argmax element/IP = 1242/7 in both the reference and B1D4.

The global phase check did not show a cycle-phase mismatch. Both runs used CYCLE_0250; the cycle had local time from 0 to 1, displacement extrema matched exactly, and the global loop-area error was only 0.107074%. Therefore the B1D4 failure is not explained by a simple output-phase or step-numbering mismatch. It appears to be a real local stress/reinjection sensitivity in the B1D4 path.

This keeps the conservative decision unchanged: B1D5 is a useful pass case, but B1D3 and B1D4 show that the early-cycle jump boundary is not robust when local stress extrema are treated as hard primary rejection variables.

11.6 B1D3 Confirmation Result

B1D3 completed successfully as an Abaqus job and was compared against the 1000-cycle reference at cycle 250. It did not pass the fixed-jump gate.

Table 18: B1D3 comparison against the 1000-cycle reference at cycle 250.

Quantity	Relative error
RF1_max	1.45771%
RF1_min	0.661665%
loop_area_abs	0.214351%
HOLE_RING_MISES_MAX	7.43637%
HOLE_RING_S11_MAX_ABS	9.13095%
HOLE_RING_SDV1_MAX	1.52456%
HOLE_RING_SDV8_MAX	9.68802%
HOLE_RING_SDV11_MAX	5.43394%

B1D3 confirms that the B1D4 behavior is not an isolated single-case artifact. The global response remains accurate, but the local hole-ring stress maximum and SDV11 response exceed the strict local threshold. The current interpretation is therefore:

> Early-cycle jumping from cycle 100 is not robust under local stress-maximum acceptance. The pass at cycle 105 is useful evidence that small jumps can work, but the non-monotonic failures at cycles 103 and 104 require a more careful local acceptance strategy or a later-start jump strategy.

11.7 Revised Local-Stress Acceptance Policy

The B1D3/B1D4/B1D5 boundary is scientifically important because it shows that smaller jumps are not always better when judged by a single local stress maximum. The robust conclusion is:

> Global hysteresis quantities are robust, but local stress-maximum metrics near the hole are non-monotonic and highly sensitive to manual state initialization and re-equilibration.

Therefore, `HOLE_RING_S11_MAX_ABS` is retained and reported, but it should not be used as the only hard pass/fail metric. The revised three-level metric strategy is:

- Primary pass/fail: `RF1` extrema, loop area, `HOLE_RING_SDV1_MAX`, `HOLE_RING_SDV11_MAX`, and `HOLE_RING_MISES_MAX`.
- Secondary/diagnostic: `HOLE_RING_S11_MAX_ABS`, single-point stress maxima, and `SDV8`.
- Robust local-field additions: mean/median, 95th percentile, or 99th percentile hole-ring stress errors from pointwise distributions.

This does not hide the local stress sensitivity. It reports it as evidence that local extrema are more restrictive and less stable than global hysteresis metrics.

The current reporting conclusion is:

> The B1 sensitivity study showed that the original jump from cycle 100 to 125 was too aggressive for the local hole-ring variables. Reducing the jump to cycle 105 gave an acceptable maximum primary error of 2.35%, but the cycle 103 and 104 cases both produced non-monotonic local stress errors controlled by `HOLE_RING_S11_MAX_ABS`. Therefore, early-cycle jumping from cycle 100 cannot yet be accepted as robust under a hard local stress-maximum criterion.

B2 and B3 remain held. The next two-job policy is therefore used for one smaller cycle-100 diagnostic jump and one later-cycle safe pilot, rather than for the original larger fixed jumps.

Table 19: Next two-job diagnostic pair submitted after the B1D3 result.

Case	Route	Purpose	PBS job
`B1D2_100_to_102_to_250`	100 $\rightarrow$ 102 $\rightarrow$ 250	cycle-100 hard-stress check	1341026.mmaster02
`B2SAFE_250_to_260_to_500`	250 $\rightarrow$ 260 $\rightarrow$ 500	later-start safe pilot	1341027.mmaster02
`B2SAFE_EQ_250_to_260_to_500`	250 $\rightarrow$ 260 $\rightarrow$ 500	smaller equilibration increment	1341028.mmaster02

The first B2SAFE submission failed after datacheck during `FIXED_JUMP_EQUILIBRATE_TO_CYCLE_0260`. Abaqus reported slow force convergence and `TOO MANY ATTEMPTS MADE FOR THIS INCREMENT`. A distinct rerun case, `B2SAFE_EQ_250_to_260_to_500`, reduced the equilibration initial increment from 1.0×10^{-6} to 1.0×10^{-8}, but it failed in the same first equilibration step after cutbacks to 6.25×10^{-10}. This is a useful result by itself: the later cycle-250 reinjection point is numerically more difficult to equilibrate than the cycle-100 cases, and it requires an explicit cycle-250 reinjection/equilibration diagnostic before B2-safe can be interpreted scientifically.

11.8 Unattended Dependency Queue

The unattended queue was reorganized into two dependency-controlled streams so that at most two 16-CPU jobs can run at once while preserving the scientific gates.

Table 20: Dependency-controlled unattended queue.

Stream	Case	PBS job	Dependency/status
A	`B1D2_100_to_102_to_250`	1341026.mmaster02	running
A	`B1D1_100_to_101_to_250`	1341029.mmaster02	`afterany:1341026`
B	`B0_250_to_500`	1341033.mmaster02	failed in reinjection equilibration
B	`B2D2_250_to_252_to_500`	1341034.mmaster02	afterok not satisfied
B	`B2D5_250_to_255_to_500`	1341035.mmaster02	afterok not satisfied
B	`B0_AUDIT_0250_INITIALIZATION_ONLY`	1341036.mmaster02	failed in equilibration

The first B0-2 exact-reinjection submission, 1341030.mmaster02, exited with status 126 because the PBS script had CRLF line endings and invoked `/bin/bash^M`. Its dependent jobs 1341031 and 1341032 did not run as meaningful solver jobs. The exact-reinjection submit writer was corrected to write LF-only PBS scripts, the remote PBS file was normalized, and Stream B was resubmitted as jobs 1341033–1341035.

The corrected B0-2 job, 1341033.mmaster02, then ran Abaqus and failed during the first `REINJECTION_EQUILIBRATE` increment with `TOO MANY ATTEMPTS MADE FOR THIS INCREMENT`. Therefore B2D2 and B2D5 correctly remained blocked by their `afterok` dependencies. The Stream B replacement, `B0_AUDIT_0250_INITIALIZATION_ONLY` job 1341036.mmaster02, also failed in its first initialization/equilibration increment after cutbacks to 6.25×10^{-11}. This means the cycle-250 problem is not a jump-size result: exact cycle-250 manual state injection cannot yet be equilibrated robustly.

11.9 Long-Weekend Dependency Extension

Because the user will be away for more than 40 hours, the unattended queue was extended with additional scientifically safe diagnostics. The queue still preserves the two-job policy: only one Stream A job and one Stream B job can run at a time, and every job requests 16 CPUs, 90 GB, and 24 hours.

Table 21: Extended long-weekend queue.

Stream	Case	PBS job	Dependency/status
A	`B1D2_100_to_102_to_250`	1341026.mmaster02	running
A	`B1D1_100_to_101_to_250`	1341029.mmaster02	`afterany:1341026`
A	`B1D2_EQ_100_to_102_to_250`	1341037.mmaster02	`afterany:1341029`
A	`B1D1_EQ_100_to_101_to_250`	1341038.mmaster02	`afterany:1341037`
B	`B0_AUDIT_0500_INITIALIZATION_ONLY`	1341039.mmaster02	running
B	`B0_AUDIT_0250_SDVINI_ONLY`	1341040.mmaster02	`afterany:1341039`
B	`B0_AUDIT_0250_SIGINI_ONLY`	1341041.mmaster02	`afterany:1341040`
B	`B0_AUDIT_0500_SDVINI_ONLY`	1341042.mmaster02	`afterany:1341041`
B	`B0_AUDIT_0500_SIGINI_ONLY`	1341043.mmaster02	`afterany:1341042`

The two `B1D*_EQ` jobs are fallback diagnostics with a smaller equilibration initial increment. They should be interpreted as numerical sensitivity checks for the cycle-100 boundary, not as authorization to submit B2/B3. The Stream B audit jobs split the manual cycle-250 and cycle-500 state injection into full, `SDVINI`-only, and `SIGINI`-only variants. This should identify whether the cycle-250/500 equilibration barrier is driven primarily by STATEV injection, stress injection, or the coupled manual restart.

The Stream B injection-only diagnostics completed quickly and all returned exit status 1. Therefore, a second B1-only EQ stream was submitted into the free slot. This keeps the cluster at the intended two active 16-core jobs while continuing to avoid B2/B3.

Table 22: Final long-weekend live and queued jobs.

Stream	Case	PBS job	Dependency/status
A	`B1D2_100_to_102_to_250`	1341026.mmaster02	running
A	`B1D1_100_to_101_to_250`	1341029.mmaster02	`afterany:1341026`
A	`B1D2_EQ_100_to_102_to_250`	1341037.mmaster02	`afterany:1341029`
A	`B1D1_EQ_100_to_101_to_250`	1341038.mmaster02	`afterany:1341037`
A	`B1D3_EQ_100_to_103_to_250`	1341044.mmaster02	`afterany:1341038`
A	`B1D4_EQ_100_to_104_to_250`	1341045.mmaster02	`afterany:1341044`
A	`B1D5_EQ_100_to_105_to_250`	1341046.mmaster02	`afterany:1341045`
C	`B1Q_EQ_100_to_106_to_250`	1341047.mmaster02	running
C	`B1S_EQ_100_to_112_to_250`	1341048.mmaster02	`afterany:1341047`
C	`B1_EQ_100_to_125_to_250`	1341049.mmaster02	`afterany:1341048`

The final live/queued solver count after this extension was 10 jobs: 2 running and 8 held by dependencies. The active requested resource load was therefore $2 \times 16 = 32$ CPU cores.

11.10 Post-Queue Check on 2026-06-07

The long-weekend queue had drained by the 2026-06-07 check: `qstat -u pr21vyci` returned no active jobs. The PBS extended-history query with `qstat -x -f` no longer exposed the listed job IDs, returning `Unknown Job Id` for 1341026, 1341029, 1341037, 1341038, 1341044–1341049, and the audit jobs 1341039–1341043. Therefore the final job assessment below is based on the completed run artifacts in the remote Stage 16N directory and the refreshed comparison CSVs, not on retained PBS accounting records.

Table 23: Final fixed-jump comparison after the dependency queue drained.

Case	Route	Status	Max primary error	Controller
`B1D1`	$100 \to 101 \to 250$	review	8.70868%	`SDV11`
`B1D1_EQ`	$100 \to 101 \to 250$	review	8.70868%	`SDV11`
`B1D2`	$100 \to 102 \to 250$	review	8.79420%	`SDV11`
`B1D2_EQ`	$100 \to 102 \to 250$	review	8.79420%	`SDV11`
`B1D3`	$100 \to 103 \to 250$	review	7.43637%	`MISES`
`B1D3_EQ`	$100 \to 103 \to 250$	review	7.43637%	`MISES`
`B1D4`	$100 \to 104 \to 250$	review	7.54718%	`SDV11`
`B1D4_EQ`	$100 \to 104 \to 250$	review	7.54718%	`SDV11`
`B1D5`	$100 \to 105 \to 250$	pass	2.34986%	`SDV1`
`B1D5_EQ`	$100 \to 105 \to 250$	pass	2.34986%	`SDV1`
`B1Q`	$100 \to 106 \to 250$	review	5.75372%	`MISES`
`B1Q_EQ`	$100 \to 106 \to 250$	review	5.75372%	`MISES`
`B1S`	$100 \to 112 \to 250$	review	7.63033%	`SDV11`
`B1S_EQ`	$100 \to 112 \to 250$	review	7.63033%	`SDV11`
`B1`	$100 \to 125 \to 250$	review	11.27770%	`SDV11`
`B1_EQ`	$100 \to 125 \to 250$	review	11.27770%	`SDV11`

The smaller-equilibration `EQ` reruns reproduced the same comparison errors as their non-`EQ` counterparts. This is important: the cycle-100 B1 jump-size trend is controlled by the physical/local state mismatch relative to the 1000-cycle reference, not by the initial equilibration increment used in the fixed-jump continuation. Under the current 5% primary-error gate, only the 100 → 105 → 250 case passes; the 100 → 106 and larger jumps remain above threshold, and the 100 → 101/102/103/104 cases still show local-state or Mises review errors.

The later-cycle fixed-jump candidates `B2`, `B2SAFE`, `B2SAFE_EQ`, `B2D2`, `B2D5`, and `B3` remain `missing_outputs`. This is expected because the cycle-250 exact-reinjection/equilibration gate failed and the unsafe dependent jobs were not allowed to produce accepted fixed-jump validations.

11.11 Robust Scalar-Metric Re-score

The completed B1/B1_EQ family was re-scored with robust scalar statistics: mean, median, 95th percentile, 99th percentile, and maximum relative error. This re-score uses the lightweight scalar metrics retained in the repository. It is not yet a full pointwise field-percentile study because complete per-integration-point fields were not retained for every B1 case. Loop errors were normalized by the full reference loop amplitude for `U1_avg` and `RF1_sum`, avoiding artificial blow-ups at zero crossings.

Table 24: Robust scalar-metric re-score for representative B1 cases.

Case	Mean	Median	p95	p99	Max	Status
`B1D1`	2.97163%	1.77699%	7.47991%	8.46293%	8.70868%	review
`B1D3`	2.78810%	1.49114%	6.93576%	7.33625%	7.43637%	review
`B1D5`	0.392238%	0.000247%	1.76313%	2.23251%	2.34986%	pass by max
`B1D5_EQ`	0.392238%	0.000247%	1.76313%	2.23251%	2.34986%	pass by max
`B1Q`	2.54352%	2.34673%	4.99948%	5.60287%	5.75372%	robust pass, max review
`B1Q_EQ`	2.54352%	2.34673%	4.99948%	5.60287%	5.75372%	robust pass, max review
`B1S`	3.65193%	3.12240%	7.50946%	7.60616%	7.63033%	review
`B1`	4.64409%	2.33580%	11.0312%	11.2284%	11.2777%	review

The robust scalar re-score supports the same conservative conclusion as the hard maximum-error gate. `B1D5` and `B1D5_EQ` are the only clean passes. `B1Q` and `B1Q_EQ` are scientifically interesting because their primary p95 error is just below 5%, but their maxima still exceed the 5% gate. Therefore they should be treated as borderline review cases unless a later full-field percentile acceptance rule is deliberately adopted.

11.12 Limitations of Manual SDVINI/SIGINI Reinjection

The later-cycle diagnostics show that the bottleneck is no longer only jump size. Manual `SDVINI/SIGINI` reinjection starts a scratch Abaqus model with injected stress and state variables, but it does not preserve the compatible displacement field, strain field, equilibrium state, solver history, or tangent/history information of the original inhomogeneous analysis. At cycle 100 this route is good enough for a narrow global-response demonstration. At cycle 250 and cycle 500 it failed before valid physical continuation in exact, `SDVINI`-only, `SIGINI`-only, and combined diagnostics.

This is a thesis-useful limitation result: manual state initialization is not robust enough for later-cycle reinjection in the plate-with-hole benchmark. Repeating B2/B3 jobs with the same scratch `SDVINI/SIGINI` route is therefore not expected to add scientific value. The next path is Stage 16N-R, a reinjection-strategy repair that preserves Abaqus' finite-element state through native restart or in-analysis continuation and applies the cycle jump only to independent material memory variables.

12 Failures and Fixes

Table 25: Important failures and fixes recorded in Stage 16N.

Issue	Observation	Fix / conclusion
Accidental serial-like run	A 22-hour run reached only 592 full cycles.	Parallel launcher corrected; later 1000-cycle reference completed.
30-CPU over-allocation	30-thread run averaged about 10 effective cores.	16 CPUs selected as production compromise.
Missing start/end email	PBS accounting showed `Mail_Points = a`.	Added `#PBS -m abe` and mail user to Stage 16N submit scripts.
CSV state-reader race	Threaded Abaqus calls attempted concurrent CSV preload.	Replaced with direct-access binary state reader.
Wrong direct-access record length	Abaqus/Intel runtime expected word-based `RECL`.	Set `RECL = 66` for 33 double values.
Postprocessing path failure	B0-1 solver completed but PBS exit status was 2.	Fixed extractor path and reran extraction manually.
B0-1 local mismatch	Global errors under 1%, local `SDV8` max error 10.9164%.	Added pointwise diagnostic and initialization-only audit.
Initialization local mismatch	Binary audit passed, but `SDV8` mismatch appeared immediately.	Concluded manual `SDVINI/SIGINI` is not a mathematically exact local restart.
Metric policy mismatch	Original adaptive table allowed `SDV8` to act as a hard controller.	Added reinjection-aware table with `SDV8` diagnostic-only.
B1 fixed-jump gate	Global B1 errors were small, but `SDV1` and `SDV11` primary local errors exceeded 10%.	Hold B2/B3 and use the two-job policy for smaller/debug B1 variants.
B1D4/B1D5 non-monotonic boundary	B1D5 passed at 2.34986%, but B1D4 failed at 9.34307% controlled by `S11`.	Accept `100 -> 105` only with diagnostic confirmation; diagnose B1D4 before B2/B3.
B1D4 pointwise diagnostic	B1D4 and the reference share the same `S11_ABS` argmax location, and median/p95 stress errors are not negligible.	Treat B1D4 as real local stress sensitivity, not a simple phase or argmax-swap artifact.
B1D3 confirmation	B1D3 also failed at 9.13095%, controlled by `S11_ABS`.	Do not generalize the B1D5 pass; treat cycle-100 jumping as not robust under hard local stress maxima.
Metric policy revision	`S11_MAX_ABS` is non-monotonic across B1D3/B1D4/B1D5.	Keep `S11` maxima as diagnostic warnings; use global, STATEV, and Mises-type metrics as primary acceptance controls.

Issue	Observation	Fix / conclusion
Completed B1 dependency queue	The drained queue shows `B1D5` and `B1D5_EQ` pass at 2.34986%, while all other completed B1-size variants remain review cases.	Treat 100 → 105 → 250 as the only currently accepted B1 fixed jump; keep B2/B3 blocked by the cycle-250 reinjection gate.
Cycle-250 reinjection gate	B2SAFE, B2SAFE_EQ, exact B0-2, and `B0_AUDIT_0250_INITIALIZATION_ONLY` all failed in their first equilibration increment.	Hold all cycle-250 fixed jumps; diagnose the cycle-250 manually injected stress/state field before rerunning B2SAFE.

13 Current Conclusion

Stage 16N has successfully produced a full 1000-cycle non-jump reference and a conservative adaptive ΔN table. The inhomogeneous plate-with-hole benchmark is scientifically useful because both global hysteresis quantities and local hole-ring state variables evolve significantly. The HPC workflow is resource-controlled with a 16-CPU production policy and explicit start/end/failure mail notifications.

The fixed-jump result is now frozen as Stage 16N-C. The cycle-jump method can work in the inhomogeneous Abaqus model, but only for a very conservative early jump: `B1D5`/`B1D5_EQ`, 100 → 105 → 250, with 2.34986% maximum primary scalar-metric error. The adaptive cycle-100 table estimated $\Delta N \approx 24$, while the accepted observed jump is $\Delta N = 5$, giving an effective safety factor of about $5/24 \approx 0.21$. Thus the thesis-safe result is that adaptive ΔN estimates must be strongly reduced when applied to the inhomogeneous plate-with-hole benchmark. Local hole-ring stress and STATEV extrema are much more restrictive than the global hysteresis response.

The important methodological limitation is exact local reinjection. B0-1 proves that the manual `SDVINI`/`SIGINI` workflow can run and reproduce the global force-displacement response well near cycle 100, but it does not reproduce local strain-like state variables exactly. The cycle-100 initialization-only audit shows that this local mismatch appears immediately during manual Abaqus initialization/equilibration, even though the binary reader is exact. The cycle-250 and cycle-500 exact, `SDVINI`-only, `SIGINI`-only, and combined diagnostic failures show that the same mechanism becomes a hard equilibration gate at later cycles. The next scientific path is therefore not more scratch state-initialized B2/B3 jobs, but Stage 16N-R: a restart-preserved material-state jumping strategy.

14 Next Work

The completed dependency queue shows that a small jump from cycle 100 can work only in a narrow window: `B1D5_100_to_105_to_250` and `B1D5_EQ_100_to_105_to_250` passed with maximum primary error 2.34986%. The surrounding smaller and larger B1 variants remain review cases, including `B1D1`, `B1D2`, `B1D3`, `B1D4`, `B1Q`, `B1S`, and `B1`. The `EQ` fallback cases did not reduce the comparison errors. B0-2 and B0-3 remain on hold as exact-local-restart claims, and B2/B3 fixed jumps are also held because the cycle-250 reinjection/equilibration gate has not passed. The next work is the Stage 16N-R repair path.

1. Freeze Stage 16N-C as a validated small-jump demonstration plus a limitation study of manual reinjection.

2. Do not submit B2/B3/B3SAFE/B4SAFE with the current scratch `SDVINI/SIGINI` route.
3. Use the robust scalar re-score as supporting evidence: only `B1D5` and `B1D5_EQ` pass the hard 5% maximum primary-error gate; `B1Q/B1Q_EQ` remain borderline review cases.
4. Treat the completed STATEV independence audit as the first Stage 16N-R artifact. Independent memory variables are `STATEV(1)`, `STATEV(2:19)`, and `STATEV(20:25)`; `STATEV(26)` is derived from alpha and `STATEV(27)` is increment-local/diagnostic.
5. Prepare a restart-enabled reference or checkpoint run because no local usable `.res/.stt/.mdl/.sim` restart files were found for the completed Stage 16N reference.
6. Run a native Abaqus restart control before any new jump: continue from checkpoint without memory overwrite and compare 250 $\to$ 500 and 500 $\to$ 1000 against the full reference.
7. Implement restart-preserved material-memory overwrite only after the native restart control passes, and modify only the independent material memory variables inside the UMAT.
8. Then retry later jumps with the preserved FE state, starting with small controlled jumps rather than the original B2/B3 queue.

15 Stage 16N-R1 Restart-Enabled Reference Submission

Stage 16N-R1 was started on 2026-06-07 to create native Abaqus restart files before any restart-preserved UMAT overwrite is attempted. Two ordinary no-jump reference jobs were generated and submitted under the two-job policy:

Table 26: Stage 16N-R1 restart-enabled reference jobs.

Case	PBS job	Abaqus job	Target	Restart checkpoints
`R1B`	`1341177.mmaster02`	`stage16n_r1b_restart_ref_250cycles`	250 cycles	100, 250
`R1A`	`1341178.mmaster02`	`stage16n_r1a_restart_ref_500cycles`	500 cycles	100, 250, 500

Both jobs request `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90gb`, `walltime=24:00:00`, and run Abaqus with `cpus=16 mp_mode=threads`. The first attempted submissions, `1341175.mmaster02` and `1341176.mmaster02`, failed before Abaqus started because the generated per-case shell runner had Windows CRLF line endings. The runner was corrected to write LF line endings explicitly, the remote copies were normalized, and the corrected jobs were submitted as `1341177.mmaster02` and `1341178.mmaster02`. At the first `qstat -f` check both corrected jobs were running.

The acceptance condition for this substage is not a cycle-jump error threshold. It is the existence of usable native Abaqus restart files from the 250-cycle and 500-cycle checkpoint references. Only after those restart sources exist should native restart continuation tests be prepared.

15.1 Stage 16N-R1 Completion Check on 2026-06-08

Both restart-enabled reference analyses completed successfully as Abaqus jobs. PBS marked the jobs with exit status 2 because the post-run status-file shell block failed after extraction; this was a scripting issue, not a solver failure. The `.sta` files for both jobs end with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`, and both logs show `1 MPI RANK x 16 THREADS`.

Table 27: Stage 16N-R1 completed job accounting.

Case	PBS job	Walltime	CPU time	CPU percent	PBS status
`R1B`	`1341177.mmaster02`	03:13:22	18:58:17	655	exit 2 after extraction
`R1A`	`1341178.mmaster02`	08:32:36	43:30:54	661	exit 2 after extraction

Native restart files now exist. The 250-cycle reference produced `.res` (29 MB), `.mdl` (50 MB), `.sim` (1.6 MB), and `.stt` (226 GB). The 500-cycle reference produced `.res` (78 MB), `.mdl` (107 MB), `.sim` (1.6 MB), and `.stt` (1.1 TB). These large `.stt` files should not be deleted until native restart continuation has been tested.

The extracted endpoint global metrics match the existing 1000-cycle reference exactly at the checked cycles. For `R1B` at cycle 250 and `R1A` at cycle 500, the shared `U1_max`, `U1_min`, `RF1_max`, `RF1_min`, and `loop_area_abs` errors are all 0%. Therefore Stage 16N-R1 achieved its immediate purpose: FE-consistent native restart source files now exist and the restart-enabled no-jump runs reproduce the existing reference metrics at the endpoints.

The next substage should be native restart continuation control with no UMAT overwrite: restart from cycle 100 and continue to 250, then restart from cycle 250 and continue to 500. Only after those native restart continuations pass should restart-preserved UMAT memory overwrite be implemented.

16 Stage 16N-R2 Native Restart Control Submission

Stage 16N-R2 was prepared on 2026-06-08 as the next gate after the restart-enabled R1 references. The purpose is to test native Abaqus restart continuation with no `SDVINI/SIGINI` scratch reinjection and no UMAT memory overwrite. This directly checks whether Abaqus can preserve the finite-element displacement, strain, equilibrium, solver-history, and material state from its own restart files.

Table 28: Stage 16N-R2 native restart control jobs.

Case	PBS job	Restart read	Continuation	Scientific status
`R2C1`	`1341284.mmaster02`	step 100, inc 53	100 → 250	pass after follow-up postprocess
`R2C2`	`1341285.mmaster02`	step 250, inc 58	250 → 500	pass after follow-up postprocess

Both corrected jobs request `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90gb`, `walltime=24:00:00`, and run Abaqus with `cpus=16 mp_mode=threads`. The first submitted R2 attempt, `1341282.mmaster02` and `1341283.mmaster02`, failed during Abaqus datacheck because the oldjob `.odb` file was not symlinked into the restart case folders. This was a restart-setup issue, not a solver result. The link script was corrected to expose the oldjob `.odb` together with `.res`, `.stt`, `.mdl`, `.sim`, and `.prt`; the corrected submissions then ran to completion.

16.1 Stage 16N-R2 Completion Check on 2026-06-08

PBS history shows that both native restart control jobs finished with `Exit_status = 1`; however, both Abaqus solver runs completed successfully. Follow-up extraction and comparison then completed on HPC, and the scientific restart-control gate passed.

Table 29: Stage 16N-R2 finished job accounting.

Case	Walltime	CPU time	CPU percent	Mem	Vmem	Finish time
R2C1	01:55:33	11:15:42	639	94374872kb	5116372kb	Mon Jun 8 08:42:46 2026
R2C2	03:43:52	20:26:04	643	94375848kb	8489656kb	Mon Jun 8 10:31:06 2026

The full PBS accounting for `R2C1` records `ctime = Mon Jun 8 06:47:01 2026`, `stime = Mon Jun 8 06:47:09 2026`, `exec_host = mnode100/0*0`, and `Exit_status = 1`. The full PBS accounting for `R2C2` records `ctime = Mon Jun 8 06:47:01 2026`, `stime = Mon Jun 8 06:47:07 2026`, `exec_host = mnode101/0*0`, and `Exit_status = 1`. The nonzero PBS status is classified as wrapper/postprocessing failure, not Abaqus restart failure.

16.2 Stage 16N-R2 Follow-up Comparison

Manual follow-up postprocessing produced `stage16n_native_restart_comparison_summary.csv` in both R2 case folders. The `R2C1` comparison at cycle 250 reports `status = pass`, `max_global_error_pct = 0`, and `max_primary_local_error_pct = 0`. The `R2C2` comparison at cycle 500 reports the same values. This confirms that native Abaqus restart can preserve the inhomogeneous finite-element state and reproduce the 1000-cycle reference exactly at the checked continuation end-points.

17 Stage 16N-R3 Restart Debug Smoke Tests

Stage 16N-R3D was run as a minimal native restart diagnostic after the R2 wrapper/postprocessing failure. The aim was to preserve the Abaqus finite-element restart state and inspect the first UMAT calls after restart before implementing any material-memory overwrite.

Table 30: Stage 16N-R3D restart debug jobs.

Case	PBS job	Restart	Target	Solver	Debug	Exit
R3D1	1341601.mmaster02	250	251	completed	pass	0
R3D2	1341602.mmaster02	500	501	completed	pass	0

PBS history was checked on 2026-06-09. Both jobs used `select=1:ncpus=16:mpiprocs=1:ompthreads=16:`, `Resource_List.ncpus = 16`, `Resource_List.mpiprocs = 1`, `Resource_List.mem = 90gb`, and `Resource_List.walltime = 24:00:00`. `R3D1` ran on `mnode100/0*0`, with `ctime = Mon Jun 8 12:35:10 2026`, `stime = Mon Jun 8 12:35:12 2026`, `mtime = Mon Jun 8 12:36:22 2026`, `resources_used.walltime = 00:01:04`, `resources_used.cput = 00:04:30`, `resources_used.cpupercent = 422`, `resources_used.mem = 1545812kb`, `resources_used.vmem = 4859968kb`, `resources_used.ncpus = 16`, and `Exit_status = 0`. `R3D2` ran on `mnode101/0*0`, with `ctime = Mon Jun 8 12:35:11 2026`, `stime = Mon Jun 8 12:35:12 2026`, `mtime = Mon Jun 8 12:36:45 2026`, `resources_used.walltime = 00:01:26`, `resources_used.cput = 00:06:41`, `resources_used.cpupercent = 461`, `resources_used.mem = 2125400kb`, `resources_used.vmem = 8032600kb`, `resources_used.ncpus = 16`, and `Exit_status = 0`.

The restart debug traces show that the first safe overwrite hook is the restart continuation step with `KINC = 0`, `TIME(1) = 0`, and `TIME(2)` equal to the checkpoint cycle within floating-point tolerance. For `R3D1`, the first restart call had `JSTEP(1) = 251`, `KINC = 0`, and `TIME(2) = 250.000000000019`. For `R3D2`, the first restart call had `JSTEP(1) = 501`, `KINC = 0`, and `TIME(2) = 499.999999999974`. This confirms that native restart exposes the restarted material state to UMAT before the first continuation increment.

The next thesis-safe implementation step is Stage 16N-R3E exact no-op overwrite. The overwrite should target only independent material memory variables, namely STATEV(1), STATEV(2:19), and STATEV(20:25). STATEV(26) and STATEV(27) should not be independently overwritten because they are derived or diagnostic state.

18 Stage 16N-R3E Exact Restart-Preserved Overwrite

Stage 16N-R3E is the first active implementation of the restart-preserved route. It is not yet a cycle-jump test. The objective is to prove that a UMAT can overwrite exact material memory inside a native Abaqus restart run without destroying the preserved displacement, strain, equilibrium, and solver state.

The prepared controls are:

Table 31: Stage 16N-R3E exact overwrite controls submitted on 2026-06-09.

Case	PBS job	Restart	Target	Overwrite trigger	Variables
R3E1	1342248.mmaster02	250	500	JSTEP(1)=251, KINC=0	STATEV(1:25)
R3E2	1342249.mmaster02	500	750	JSTEP(1)=501, KINC=0	STATEV(1:25)

The UMAT hook reads exact per-element/per-integration-point state from the restart-source .odb, converted to a direct-access binary table. It overwrites only independent material-memory variables, STATEV(1:25). It does not table-overwrite STATEV(26) or STATEV(27); STATEV(26) remains derived by the constitutive update and STATEV(27) remains increment-local diagnostic state.

The first R3E submission attempt, 1342246.mmaster02 and 1342247.mmaster02, failed before the solve because Abaqus could not resolve scratch=. from the long case directory. This was an infrastructure issue, not a UMAT or restart-state result. The runner was corrected to use the short PBS scratch directory `$PBS_JOBDIR`. Corrected submissions 1342248.mmaster02 and 1342249.mmaster02 started at Tue Jun 9 06:09:58 2026. Both request select=1:ncpus=16:mpiprocs=1:ompth walltime=24:00:00, and both datachecks compiled the overwrite UMAT successfully before entering the production solve.

Both corrected jobs completed Abaqus successfully. PBS history reports Exit_status=1 and Stageout_status=1 for both jobs, but the solver files show THE ANALYSIS HAS COMPLETED SUCCESSFULLY in the .sta files and THE ANALYSIS HAS BEEN COMPLETED in the .msg files. The nonzero PBS status is therefore a stage-out or wrapper artifact, not a solver failure. Final accounting was:

Table 32: Stage 16N-R3E final PBS accounting.

Job	Case	Walltime	CPU time	CPU %	Memory	Host
1342248	R3E1	04:06:17	22:36:56	639	94375880kb	mnode101
1342249	R3E2	04:10:00	22:43:25	643	94375816kb	mnode102

The comparison against the reference was exact at the target cycles:

Table 33: Stage 16N-R3E exact overwrite comparison results.

Case	Target cycle	Status	Max global error	Max primary local error
R3E1	500	pass	0%	0%
R3E2	750	pass	0%	0%

R3E1 was compared against the default Stage 16N 1000-cycle pilot reference at cycle 500. R3E2 was compared against the parallel 1000-cycle reference because the default pilot comparison table did not contain cycle 750. The exact zero-error result confirms that modifying only independent material-memory variables inside a native restart does not destroy the preserved finite-element equilibrium state.

The current thesis-safe conclusion is therefore:

> Native Abaqus restart reproduced the reference exactly, proving that the inhomogeneous FE state is restartable when Abaqus preserves the displacement, strain, and equilibrium state. The next step is therefore to apply state changes inside the restarted analysis, rather than attempting scratch initialization through `SDVINI/SIGINI`.

19 Stage 16N-R3J Restart-Preserved Jump Tests

Stage 16N-R3J is the first nonzero repaired cycle-jump workflow after the R3E exact-overwrite pass. The finite-element displacement, strain, equilibrium, and solver state are still preserved by native Abaqus restart. The UMAT modifies only independent material memory, `STATEV(1:25)`, at the restart continuation call with `KINC=0`. The state modification is a cycle-space extrapolation,

$$\texttt{STATEV\_jump} = \texttt{STATEV\_base} + \Delta N \frac{d\texttt{STATEV}}{dN}.$$

The active production submissions are:

Table 34: Stage 16N-R3J active restart-preserved jump jobs submitted on 2026-06-10.

Case	PBS job	Restart	Slope pair	Material jump	Target
R3J1	1342923.mmaster02	250	100 → 250	250 → 255	500
R3J2	1342924.mmaster02	500	250 → 500	500 → 505	750

The originally desired one-cycle slope pairs, 249 → 250 and 499 → 500, were not available in the R1A restart-source `.odb` as stress/SDV field-output frames. The active corrected jobs therefore use the nearest available endpoint field-output pairs ending at the restart checkpoints: 100 → 250 for R3J1 and 250 → 500 for R3J2. This slope choice is less local than the initial ideal one-cycle slope, so it must be reported with the R3J results.

Several setup-only attempts were needed before the active production jobs. Jobs `1342915` and `1342916` failed because an upload quoting issue populated incorrect remote directories. Jobs `1342917` and `1342918` failed because cycles 249 and 499 had no stress/SDV field-output frames. Jobs `1342919` and `1342920` failed because the cluster Python rejected `from __future__ import annotations`. Jobs `1342921` and `1342922` failed because the older `pathlib.write_text` implementation rejected the `newline` keyword. These were setup and compatibility failures before the production solve, not scientific R3J failures.

The active corrected submissions, `1342923.mmaster02` and `1342924.mmaster02`, passed state extraction, extrapolated-state creation, UMAT compile/link, and Abaqus datacheck. At the first live resource check both were running under the standard production setup: `select=1:ncpus=16:mpipr` `walltime=24:00:00`, and Abaqus `cpus=16 mp_mode=threads`. `1342923` ran on `mnode101/0*0`; `1342924` ran on `mnode102/0*0`.

The R3J comparison classifies results using global RF/U/loop metrics, primary local metrics `HOLE_RING_MISES_MAX`, `HOLE_RING_SDV1_MAX`, `HOLE_RING_SDV8_MAX`, and `HOLE_RING_SDV11_MAX`, and diagnostic-only `HOLE_RING_S11_MAX_ABS`. The pass threshold is `max_primary_local_error_pct <= 5`; 5–10% is review; larger than 10% or solver instability is fail.

19.1 R3J Final Check on 2026-06-11

On 2026-06-11, PBS history showed that both active R3J jobs had finished. PBS reported `Exit_status=1` and `Stageout_status=1` for both jobs, but the Abaqus solver runs completed successfully. In both cases, the `.sta` file ends with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`. The nonzero PBS status came from the wrapper comparison step using Windows-style reference paths during the original job, not from solver failure.

Table 35: Stage 16N-R3J final PBS accounting.

Job	Case	Walltime	CPU time	CPU %	Memory	Host
`1342923`	R3J1	04:07:53	22:42:03	637	94375924kb	`mnode101`
`1342924`	R3J2	04:10:06	22:44:30	646	94377920kb	`mnode102`

Both jobs requested `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90gb` with `walltime=24:00:0` R3J1 started at Wed Jun 10 16:58:01 2026 and finished at Wed Jun 10 21:05:59 2026. R3J2 started at Wed Jun 10 16:58:00 2026 and finished at Wed Jun 10 21:08:11 2026.

The comparison was rerun manually on HPC with Linux reference paths. R3J1 was compared against the pilot 1000-cycle reference at cycle 500, and R3J2 was compared against the parallel 1000-cycle reference at cycle 750. Both restart-preserved jump tests passed exactly:

Table 36: Stage 16N-R3J restart-preserved jump comparison results.

Case	Target cycle	Status	Max global error	Max primary local error	Diagnostic S11 error
R3J1	500	pass	0%	0%	0%
R3J2	750	pass	0%	0%	0%

This is the first successful nonzero repaired cycle-jump gate for the inhomogeneous plate-with-hole benchmark. Within the checked metrics, native restart plus independent material-memory extrapolation preserved the FE state and reproduced the reference exactly after the +5 material-state jump.

19.2 R3J Zero-Error Audit

Because the R3J result is exactly zero for a nonzero material-state extrapolation, a targeted audit was performed on 2026-06-11 before moving to larger jumps. The audit confirms that the comparison did not accidentally compare a reference file against itself. The R3J1 jump metrics CSV and pilot reference metrics CSV had different inode numbers, sizes, and SHA-256 hashes. The R3J2 jump metrics CSV and parallel reference metrics CSV were likewise distinct.

The extrapolated state files also differ from the base restart states. For R3J1, all 25,184 integration-point records changed between cycle 250 and the extrapolated cycle-255 state; 478,496 state values changed, with maximum absolute delta 0.527609507243. For R3J2, all 25,184 records changed between cycle 500 and the extrapolated cycle-505 state; 478,496 state values changed, with maximum absolute delta 0.294372863770.

The UMAT overwrite trace was present in the Abaqus `.dat` files, not the `.msg` files. R3J1 printed `STAGE16N_R3J_OVERWRITE` at `KSTEP=251`, `KINC=0`, and `TIME(2)=250.000000000019`. R3J2 printed the same marker at `KSTEP=501`, `KINC=0`, and `TIME(2)=499.999999999974`. This confirms that the jump was applied once at the intended native-restart continuation hook. The generator was corrected so future wrapper logs grep the `.dat` file for this marker.

The endpoint cycle accounting was also correct. R3J1 produced 250 continuation rows, cycles 251–500, and compared against reference cycle 500. R3J2 produced 250 continuation rows, cycles 501–750, and compared against reference cycle 750. The comparison details for both cases report zero error in `U1`, `RF1`, loop area, `HOLE_RING_MISES_MAX`, `HOLE_RING_SDV1_MAX`, `HOLE_RING_SDV8_MAX`, `HOLE_RING_SDV11_MAX`, and diagnostic `HOLE_RING_S11_MAX_ABS`.

The thesis-safe interpretation is therefore:

> The restart-preserved cycle-jump strategy was successfully demonstrated for nonzero +5 cycle jumps. Both jumps, from cycle 250 to 255 and from cycle 500 to 505, completed successfully in Abaqus and reproduced the corresponding full reference endpoints with zero global and primary local error. This confirms that the previous reinjection problem was not a limitation of the material model or the inhomogeneous plate-with-hole geometry, but of the scratch `SDVINI/SIGINI` initialization strategy.

19.3 R3J +10 Cases Completed

The next conservative fixed-jump cases, R3J3 and R3J4, were submitted on 2026-06-11 and completed successfully. These jobs test whether the restart-preserved material-memory jump remains exact when the extrapolated material-state offset is doubled from +5 cycles to +10 cycles.

Table 37: Stage 16N-R3J +10 restart-preserved jump cases.

Case	PBS job	Restart	Slope pair	Material jump	Target
`R3J3`	`1344231.mmaster02`	250	100 → 250	250 → 260	500
`R3J4`	`1344232.mmaster02`	500	250 → 500	500 → 510	750

PBS history reported `Exit_status=0` for both jobs, while `Stageout_status=1` remained present. The Abaqus solver status is therefore successful, and the stage-out warning is treated as a file-transfer/accounting issue rather than a solver failure. The `.sta` files for both jobs end with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`.

Table 38: Stage 16N-R3J +10 final PBS accounting.

Job	Case	Walltime	CPU time	CPU %	Memory	Host
`1344231`	R3J3	04:08:42	22:51:08	646	94375928kb	`mnode101`
`1344232`	R3J4	04:07:59	22:41:15	656	94377792kb	`mnode102`

Both jobs requested `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90gb` with `walltime=24:00:0` R3J3 started at Thu Jun 11 09:55:29 2026 and finished at Thu Jun 11 14:04:16 2026. R3J4 started at Thu Jun 11 09:55:29 2026 and finished at Thu Jun 11 14:03:34 2026.

Table 39: Stage 16N-R3J +10 comparison results.

Case	Target cycle	Status	Max global error	Max primary local error	Diagnostic S11 error
R3J3	500	pass	0%	0%	0%
R3J4	750	pass	0%	0%	0%

The +10 results preserve the same zero-error endpoint behavior observed in the +5 cases. Within the current comparison metrics, the restart-preserved method has now passed nonzero fixed jumps of +5 and +10 cycles at both restart checkpoints. This strengthens the Stage 16N-R conclusion: native Abaqus restart plus UMAT material-state overwrite solves the fragile scratch reinjection problem seen with `SDVINI/SIGINI`.

A targeted +10 audit was performed because the error remains exactly zero. The audit checks passed:

- The R3J3/R3J4 jump metrics are distinct from their reference metrics by inode, file size, and SHA-256 hash, so the jobs were not self-compared.
- The extrapolated state differs from the checkpoint base state for all 25,184 integration-point records in both jobs. R3J3 changed 528,728 of 831,072 numeric state/stress values, with maximum absolute delta 1.0552190144856723. R3J4 changed 521,304 of 831,072 numeric values, with maximum absolute delta 0.5887457275390595.
- The UMAT overwrite trace appears in the production `.dat` files at `KSTEP=251` for R3J3 and `KSTEP=501` for R3J4.
- Continuation cycle accounting is correct: R3J3 contains cycles 251–500 and R3J4 contains cycles 501–750.
- Endpoint comparisons target the intended reference cycles, 500 and 750.

The exact zero-error +10 result is therefore credible under the current audit. The scientific caution is that broad predictive acceleration should not be claimed until larger jumps reveal the accuracy boundary or continue to pass under increasingly aggressive skips. The recommended progression is +20 next, specifically R3J5: 250 → 270 → 500 and R3J6: 500 → 520 → 750. These jobs should not be submitted until the `/home` heavy-file cleanup has completed, R3J3/R3J4 heavy outputs have been moved to `/scratch`, and the next PBS scripts run Abaqus directly in `/scratch`.

19.4 R3J +20 Preparation and Cleanup-Gated Submission

On 2026-06-11, the R3J generator was extended with the next controlled escalation pair, R3J5 and R3J6. The new cases use the same restart-preserved UMAT overwrite workflow and the same endpoint comparison targets as the +5 and +10 studies:

Table 40: Stage 16N-R3J +20 cleanup-gated jump cases.

Case	Restart checkpoint	Slope pair	Material jump	Target cycle	Status
R3J5	250	100 → 250	250 → 270	500	prepared, waiting for cleanup
R3J6	500	250 → 500	500 → 520	750	prepared, waiting for cleanup

The case folders and PBS scripts were uploaded to the HPC repository, but the jobs were not submitted immediately. Instead, the cleanup-gated watchdog `~/submit_r3j_plus20_after_cleanup.sh` was installed and started as PID 653015. The watchdog checks that offload processes have finished, no PBS jobs are active, `/home` usage is at most 75%, `/scratch` usage is at most 90%, and no Abaqus heavy files larger than 10 GB remain in the `/home` project tree. Only then will it submit R3J5 and R3J6 and write the marker `.r3j_plus20_auto_submitted`. At installation time the observed storage gate was still active: `/home` was 76%, `/scratch` was 74%, and the offload processes were still running.

On 2026-06-12, the general storage gate had improved to `/home`=71% and `/scratch`=74%, but the watchdog still correctly blocked submission because three heavy `.stt` files larger than 10 GB remained in the `/home` project tree: the R2C1 native restart `.stt`, the R3J3 +10 `.stt`, and the R3J4 +10 `.stt`. A focused offload process, `/tmp/offload_remaining_heavy_for_r3j_plus20.py`, was started as PID 1878791 to copy those blockers to `/scratch/pr21vyci/home_offload/...`, verify file sizes, delete the `/home` originals, and replace them with symlinks. The cleanup-gated watchdog was restarted as PID 1877723 and remains responsible for submitting R3J5/R3J6 once this final blocker list is cleared.

The focused offload completed successfully, reducing `/home` to 64% at watchdog submission time. The watchdog then submitted R3J5 and R3J6 automatically at approximately Fri Jun 12 06:52 CEST 2026:

Table 41: Stage 16N-R3J +20 final PBS accounting.

Job ID	Case	Walltime	CPU time	CPU percent	Memory	Host
1344614	R3J5	04:13:13	22:48:35	621	94375888kb	mnode100
1344615	R3J6	04:07:32	22:32:31	641	94375872kb	mnode101

Both jobs completed with PBS `Exit_status=0` and `Stageout_status=1`. R3J5 started at Fri Jun 12 06:52:29 2026 and finished at 11:05:48. R3J6 started at Fri Jun 12 06:52:31 2026 and finished at 11:00:09. The `.sta` files for both jobs end with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`.

Table 42: Stage 16N-R3J +20 comparison results.

Case	Target cycle	Status	Max global error	Max primary local error	Diagnostic S11 error
R3J5	500	pass	0%	0%	0%
R3J6	750	pass	0%	0%	0%

The UMAT overwrite trace confirms that the state overwrite fired at the intended restart continuation hook: `KSTEP=251` for R3J5 and `KSTEP=501` for R3J6, with 9 diagnostic trace lines in each case. The Stage 16N-R conclusion is therefore stronger again: the restart-preserved UMAT material-state overwrite workflow has now passed nonzero +5, +10, and +20 state jumps. This supports the main repair conclusion that native Abaqus restart removes the mechanical-state reinjection instability observed in the SDVINI/SIGINI scratch route.

19.5 R3J +20 Audit

The +20 audit found one important qualification. R3J5 and R3J6 were not self-compared: their cycle-metric files and reference metric files are separate files with different SHA-256 hashes. The generated extrapolated-state summaries report 25,184 element/integration-point records, material-state cycles 270 and 520, and the intended slope pairs 100–250 and 250–500. The overwrite traces confirm that the UMAT hook fired at the first continuation increment, and the comparison endpoints are the intended cycles 500 and 750.

However, the current continuation decks do not yet skip Abaqus load-cycle steps. R3J5 restarts from cycle 250 and begins the continuation deck at `CYCLE_0251`; its extracted metrics cover cycles 251–500. R3J6 restarts from cycle 500 and begins at `CYCLE_0501`; its extracted metrics cover cycles 501–750. Therefore the present evidence should be described as a stable restart-preserved material-state jump/overwrite validation, not yet as proof of computational acceleration by solving fewer load cycles. The next acceleration-safe generator should start the continuation after the jumped cycle label, or otherwise provide a separate accounting that proves fewer solved load cycles between checkpoint and endpoint.

The controlled +50 cases R3J7 and R3J8 were prepared locally for later use, but they inherit the same continuation-step pattern. They should not be submitted as acceleration evidence until this deck-generation issue is fixed. They may still be useful as larger material-state perturbation tests if that is the explicit goal.

19.6 R4J Scratch-Based +50 Cycle-Skip Setup

The next acceleration-safe test pair was therefore generated under new case names, R4J1 and R4J2. These cases preserve the same native restart and UMAT material-state overwrite strategy, but their continuation step labels begin after the jumped material-state cycle:

Table 43: Stage 16N-R4J scratch-based +50 cycle-skip cases.

Case	Restart checkpoint	Material-state jump	Solved continuation cycles	Endpoint
R4J1	250	250 → 300	301–500	500
R4J2	500	500 → 550	551–750	750

The generated R4J1 deck starts with `*STEP, NAME=CYCLE_0301`; the generated R4J2 deck starts with `*STEP, NAME=CYCLE_0551`. The UMAT overwrite trigger remains the first restart-continuation step, `JSTEP(1)=251` for R4J1 and `JSTEP(1)=501` for R4J2, because that is where Abaqus enters the first new step after reading the native checkpoint.

To avoid repeating the HPC `/home` storage issue, the R4J pair was submitted through the scratch watchdog `watch_cleanup_and_submit_stage16n_scratch_jobs.sh`. This script waits for ongoing offload work to finish, verifies `/home` and `/scratch` usage, checks that no large Abaqus heavy files remain in the `/home` project tree, stages only the required case folders, helper scripts, reference CSV files, and restart-source symlinks to `/scratch/pr21vyci/stage16n_scratch_runs_r4j_v` submits PBS wrappers from the `/home` case folders, runs Abaqus in the scratch case directories, and copies back only lightweight evidence. The final submission required the same queue directive used by accepted Stage 16N jobs, `#PBS -q entry_teachingq`. With that fix, R4J1 and R4J2 were submitted on Sat Jun 13 06:01 CEST 2026 as jobs `1344946.mmaster02` and `1344947.mmaster02`.

19.7 R4J +50 True Cycle-Skip Result

Both R4J jobs completed successfully as Abaqus analyses. PBS reported `Exit_status=0` for both jobs, and the `.sta` files end with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`. The R4J1 job ran on `mnode100` from Sat Jun 13 06:01:25 to 10:20:56 CEST 2026, using 04:19:28 walltime, 17:56:39 CPU time, 571 CPU percent, and `94377788kb` memory. The R4J2 job ran on `mnode101` from Sat Jun 13 06:01:26 to 10:12:41 CEST 2026, using 04:11:10 walltime, 17:28:03 CPU time, 568 CPU percent, and `94372048kb` memory.

Table 44: Stage 16N-R4J true +50 cycle-skip comparison results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error
R4J1	500	fail	1.8127809%	14.384123%	9.4269674%
R4J2	750	fail	1.7938482%	14.426805%	11.71291%

The failed comparison status is an accuracy result, not a solver-stability result. In R4J1, the largest primary local error is `HOLE_RING_SDV8_MAX` at 14.384123%; `HOLE_RING_MISES_MAX` and `HOLE_RING_SDV11_MAX` also exceed 10%. In R4J2, the largest primary local error is `HOLE_RING_MISES_MAX` at 14.426805%. The global force-displacement metrics remain below about 1.9% in both cases, showing again that the local hole-ring metrics control the safe jump size.

This result provides the first true cycle-skip accuracy boundary for the repaired restart route. R3J +5, +10, and +20 validated stable restart-preserved material-state overwrite, but R4J is the first deck family that also skips solved Abaqus load cycles after the material-state target.

Therefore the thesis-safe conclusion is that native Abaqus restart plus UMAT-side material-memory overwrite solves the mechanical reinjection instability of the `SDVINI/SIGINI` scratch route, while a true +50 skipped-cycle acceleration is outside the current safe range for this inhomogeneous benchmark and linear state extrapolation. The next useful study should bracket the boundary with smaller true skips, for example +30 or +40, using the R4J continuation-step pattern.

Lightweight R4J evidence was copied into the repository. Heavy Abaqus result and restart files, including `.stt`, `.odb`, `.res`, `.sim`, `.mdl`, `.prt`, large `.msg`, large `.dat`, `state.bin`, and large `state.csv` files, are intentionally excluded from GitHub and kept in scratch/offload storage.

19.8 R4J +20 True-Skip Refinement Prepared

After the +50 accuracy failure, the next refinement pair was prepared as true-skip +20 rather than a larger jump. R4J3 restarts from cycle 250, overwrites material memory to cycle 270, and starts the solved continuation at `CYCLE_0271`; R4J4 restarts from cycle 500, overwrites material memory to cycle 520, and starts the solved continuation at `CYCLE_0521`. Thus these cases skip cycles 251–270 and 501–520, respectively, unlike the R3J overwrite-validation cases that still solved the intermediate cycles.

Table 45: Stage 16N-R4J true-skip +20 refinement cases prepared on 2026-06-13.

Case	Restart checkpoint	Material-state jump	Solved continuation cycles	Endpoint
R4J3	250	250 → 270	271–500	500
R4J4	500	500 → 520	521–750	750

The dedicated scratch watchdog `watch_cleanup_and_submit_stage16n_r4j_plus20_scratch_jobs.sh` was created to stage only the required files into `/scratch/pr21vyci/stage16n_scratch_runs_r4j_plus20/Aba` submit PBS wrappers from `/home`, run Abaqus in `/scratch`, and copy back only lightweight evidence. After eduVPN/DNS access was restored, the cases were submitted as PBS jobs `1344960.mmaster02` and `1344961.mmaster02`. Both Abaqus analyses completed successfully, and both `.sta` files end with `THE ANALYSIS HAS COMPLETED SUCCESSFULLY`.

Table 46: Stage 16N-R4J true-skip +20 refinement results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error
R4J3	500	pass	0.39923577%	3.9830029%	0.1398007%
R4J4	750	fail	2.0853964%	14.147994%	13.256425%

The R4J3 earlier-window case passes the current primary-local 5% gate. Its controlling primary-local metric is `HOLE_RING_SDV1_MAX`. The R4J4 later-window case fails the same gate, controlled by `HOLE_RING_MISES_MAX`; the diagnostic `HOLE_RING_S11_MAX_ABS` error also rises to 13.256425%.

The first true cycle-skip refinement therefore showed branch-dependent behavior. A +20 skip from cycle 250 to 270 passed the accuracy criterion, whereas the corresponding +20 skip from cycle 500 to 520 failed due to local hole-ring error. Therefore, the restart-preserved true-skip machinery is functional, but the safe jump size must be chosen adaptively and locally rather than assumed constant across the simulation. The current branch-specific bracket is:

Table 47: Stage 16N-R4J branch-specific true-skip bracket after +20 refinement.

Branch	Known pass	Known fail	Current bracket
Base cycle 250	+20	+50	Safe ΔN is between 20 and 50
Base cycle 500	none yet	+20 and +50	Safe ΔN is below 20

This is scientifically useful because later cycles are not automatically safer in the inhomogeneous model; local hole-ring variables still control the acceptable jump size. The evidence types should be reported separately: R3J validates restart-preserved overwrite but not true skipping; R4J +50 proves the true-skip machinery runs but fails accuracy; R4J +20 reveals branch-dependent accuracy. This supports the main adaptive-cycle-jump argument that fixed jump sizes are not reliable and the acceptable ΔN must follow local state evolution near the hole.

19.9 R4J Branch-Specific Next Pair Prepared

The next pair refines the two branches separately. R4J5 tests a +35 skip in the 250 branch, between the known +20 pass and +50 fail. R4J6 tests a +10 skip in the 500 branch, below the known +20 fail.

Table 48: Stage 16N-R4J branch-specific next jobs prepared after the +20 mixed result.

Case	Restart checkpoint	Material-state jump	Solved continuation cycles	Endpoint
R4J5	250	250 $\rightarrow$ 285	286–500	500
R4J6	500	500 $\rightarrow$ 510	511–750	750

The generated R4J5 deck starts with `*STEP, NAME=CYCLE_0286`; the generated R4J6 deck starts with `*STEP, NAME=CYCLE_0511`. After R4J5/R4J6 complete, the decision rule is branch-specific: if R4J5 passes, test +42 or +45 in the 250 branch; if R4J5 fails, test +27 or +30. If R4J6 passes, test +15 in the 500 branch; if R4J6 fails, test +5.

19.10 R4J Branch-Specific Refinement Result

The branch-specific R4J5/R4J6 pair was checked from PBS history and scratch evidence on 2026-06-15. Both production solver jobs completed successfully with `Exit_status=0`; the earlier scratch wrapper attempts `1345009` and `1345010` exited immediately because the restart source was missing in the first staging attempt, while the completed production jobs were `1345011.mmaster02` and `1345012.mmaster02`.

Table 49: Stage 16N-R4J branch-refinement job resource history.

Case	PBS job	Node	Walltime	CPU time	CPU percent	Memory
R4J5	1345011	mnode100	04:41:47	19:26:19	574	94375932 kb
R4J6	1345012	mnode101	05:13:59	21:45:58	563	94371840 kb

Table 50: Stage 16N-R4J branch-refinement comparison results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error
R4J5	500	review	0.00016982914%	6.9643175%	0.00027322599%
R4J6	750	review	0.31107055%	7.2880782%	1.323401%

The branch-refinement jobs therefore solved robustly but did not pass the current 5% primary-local accuracy gate. R4J5, the +35 jump from the cycle-250 branch, is close to the gate and has nearly exact global and diagnostic S11 response; its controlling local metric is `HOLE_RING_SDV1_MAX`. R4J6, the +10 jump from the cycle-500 branch, still exceeds the local gate with `HOLE_RING_SDV8_MAX` controlling.

The updated branch-specific bracket is: for the cycle-250 branch, +20 passed, +35 is review/fail against the 5% gate, and +50 failed; for the cycle-500 branch, +10, +20, and +50 all exceeded the local gate. The next useful tests are therefore below +35 for the 250 branch, for example +27 or +30, and below +10 for the 500 branch, for example +5.

19.11 R4J Next Refinement Pair Submitted

The next true-skip refinement pair was prepared from the merged HPC branch and submitted from scratch after verifying the first solved continuation steps. R4J7 tests the cycle-250 branch with a +30 skip, `250 -> 280 -> 500`, solving cycles 281–500. R4J8 tests the cycle-500 branch with a +5 skip, `500 -> 505 -> 750`, solving cycles 506–750. The generated decks begin with `*STEP, NAME=CYCLE_0281` and `*STEP, NAME=CYCLE_0506`, respectively.

The report interpretation remains: R3J is overwrite validation only, R4J is true cycle skipping, +50 was computationally stable but inaccurate, +20 was branch-dependent, and +35/+10 both exceeded the 5% local-state tolerance. The current submitted pair therefore tests +30 on the 250 branch and +5 on the 500 branch.

19.12 R4J Submitted Pair Completed

The submitted R4J7/R4J8 pair finished on 2026-06-15 with `Exit_status=0` for both jobs. The run evidence was copied into the repository under the corresponding restart-jump case directories:

- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_control/restart_jump_cases/R4J7_250_to_280_solve_281_to_500/`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_control/restart_jump_cases/R4J8_500_to_505_solve_506_to_750/`

Table 51: Stage 16N-R4J submitted-pair PBS history.

Case	PBS job	Node	Walltime	CPU time	CPU percent	Memory
R4J7	1345562	mnode100	04:41:23	19:50:24	557	94375932 kb
R4J8	1345563	mnode101	05:11:31	21:49:48	592	94371840 kb

Table 52: Stage 16N-R4J submitted-pair comparison results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error
R4J7	500	fail	1.1886389%	11.829104%	0.92936729%
R4J8	750	fail	1.4217578%	13.598377%	13.011616%

Both submitted jumps solved robustly but still exceed the current 5% primary-local gate. However, the result should not yet be interpreted as a clean monotonic safe-jump bracket. R4J7 uses a smaller true skip than R4J5, but its local error is larger: R4J5 (+35) gave 6.9643175%, while R4J7 (+30) gave 11.829104%. Likewise, R4J8 (+5) is worse than R4J6 (+10): 13.598377%

versus 7.2880782%. This can occur when the acceptance metric is a maximum over highly local hole-ring state variables, but it is also a setup-audit warning.

The worst-row audit shows that the controlling quantity changes across the branch tests. R4J5 was controlled by `HOLE_RING_SDV1_MAX`, whereas R4J7 is controlled by `HOLE_RING_SDV8_MAX`. R4J6 was controlled by `HOLE_RING_SDV8_MAX`, whereas R4J8 is controlled by `HOLE_RING_MISES_MAX`, with a diagnostic `HOLE_RING_S11_MAX_ABS` error of 13.011616%. The generated decks and overwrite traces remain consistent with the intended true-skip phase: R4J7 starts at `CYCLE_0281` and overwrites at `JSTEP(1)=251`; R4J8 starts at `CYCLE_0506` and overwrites at `JSTEP(1)=501`. The exported comparison details confirm endpoint comparisons against cycle 500 and cycle 750, respectively, but they do not preserve the element/integration-point identity of each local maximum.

19.13 R4E Exact-Target True-Skip Diagnostics Prepared

Before running another extrapolated refinement pair, exact-target true-skip controls were prepared to separate extrapolation error from restart/deck/comparison error. R4E1 restarts from cycle 250, overwrites the exact reference material state at cycle 280, and solves cycles 281–500. R4E2 restarts from cycle 500, overwrites the exact reference material state at cycle 505, and solves cycles 506–750. These controls keep the R4 true-skip solved-step accounting, but remove the linear extrapolation approximation.

Table 53: Stage 16N-R4E exact-target true-skip diagnostics prepared.

Case	Restart	Exact overwrite target	Solved continuation	Expected first step
R4E1	250	280	281–500	`CYCLE_0281`
R4E2	500	505	506–750	`CYCLE_0506`

The first R4E setup attempt showed that the exact target state cannot be extracted directly from the original restart-source ODBs: cycle 280 had no usable field-output frame in the cycle-250 source, and cycle 505 is beyond the cycle-500 source. The R4E generator was therefore corrected to create a short native-restart exact-target source solve before the true-skip deck. For R4E1 this auxiliary source solve covers cycles 251–280; for R4E2 it covers cycles 501–505. The extracted exact target state is then reinjected into the main true-skip deck, which still starts at `CYCLE_0281` for R4E1 and `CYCLE_0506` for R4E2. The corrected controls were submitted from scratch on 2026-06-15 as PBS jobs `1345655.mmaster02` and `1345656.mmaster02`.

19.14 R4E Exact-Target True-Skip Diagnostics Completed

The corrected R4E jobs finished with `Exit_status=0`. PBS history and copied lightweight evidence confirm that both the exact-target source solve and the main true-skip solve completed successfully. However, both cases still fail the primary-local acceptance gate.

Table 54: Stage 16N-R4E completed job resource history.

Case	PBS job	Host	Walltime	CPU time	CPU percent	Memory
R4E1	1345655	mnode100	05:08:21	22:15:38	593	94375852 kb
R4E2	1345656	mnode101	05:17:40	22:34:55	598	94371840 kb

Table 55: Stage 16N-R4E exact-target comparison results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error	Controlling
R4E1	500	fail	1.1886389%	11.829104%	0.92936729%	`HOLE_RING`
R4E2	750	fail	1.4217578%	13.598377%	13.011616%	`HOLE_RING`

The R4E errors are identical to the preceding R4J7/R4J8 comparison summaries, even though R4E was meant to replace the extrapolated jump state with an exact target state. A follow-up audit showed that the retained R4E exact-target CSV files are byte-identical to the `state.csv` files handed to the main true-skip decks. R4E1 used a 25184-row exact cycle-280 CSV with SHA-256 `4facbc0e87caf9c63febb31e51db819308cd496244ec33dabe79ff9a6ed2d3ff`; R4E2 used a 25184-row exact cycle-505 CSV with SHA-256 `b9254d54f41612772a55ee103b42db30734dcc8115d` These exact-target CSVs are not the same as the earlier R4J extrapolated states: the R4E1–R4J7 maximum absolute state difference is 70.03573608398443 in `S1`, and the R4E2–R4J8 maximum absolute state difference is 30.373046875 in `S1`. This rules out the simple explanation that R4E accidentally consumed the old extrapolated state file.

The overwrite traces are also phase-consistent at the diagnostic level: R4E1 prints overwrite records at `KSTEP=251`, and R4E2 prints overwrite records at `KSTEP=501`. The trace output is intentionally sparse diagnostic output rather than a complete count of every integration point. The heavy binary `state.bin` files were not retained in the lightweight result directories after cleanup, so the durable repository audit is CSV-based.

The current interpretation is therefore stronger and more specific than the initial R4E setup warning. The evidence suggests that material-only overwrite may be insufficient for a true skipped-cycle restart, because Abaqus' native restart still carries the checkpoint-cycle mechanical/internal restart state. In other words, an exact material state at cycle 280 or 505 may not be enough if the global displacement-equilibrium path, solver history, and other native restart data still come from cycle 250 or 500. The current evidence does not support continuing directly to R4J9/R4J10.

19.15 R4F Full-Target Native Restart Controls Prepared

To test the material-only overwrite hypothesis directly, R4F full-target restart controls were prepared. Each case first creates the target-cycle state with a short native restart source solve, then restarts Abaqus from that newly generated target-cycle restart file set and solves the remaining cycles without SDVINI/SIGINI and without the R4J overwrite UMAT.

Table 56: Stage 16N-R4F full-target native restart controls prepared.

Case	Source solve	Continuation solve	UMAT overwrite
R4F1	250–280	281–500	none
R4F2	500–505	506–750	none

If R4F passes while R4E fails, the failure mechanism is consistent with material-only overwrite being incomplete: the full Abaqus target restart state is needed. If R4F also fails, the remaining suspect becomes the native-restart continuation, comparison mapping, or another setup-level inconsistency. Heavy Abaqus files remain in scratch; only lightweight evidence, comparison CSVs, status notes, selected-cycle CSVs, `.sta` files, PBS stdout, and small logs should be copied into the repository.

The R4F controls were submitted on 2026-06-16 after merging the prepared files into `copilot/curved-cepha` R4F1 was PBS job `1347842.mmaster02` on `mnode101`; R4F2 was PBS job `1347843.mmaster02` on `mnode102`. Both jobs completed with `Exit_status=0`.

Table 57: Stage 16N-R4F completed job resource history.

Case	PBS job	Host	Walltime	CPU time	CPU percent	Memory
R4F1	1347842	mnode101	03:31:18	19:57:42	649	94371840 kb
R4F2	1347843	mnode102	03:30:05	19:05:48	635	94375936 kb

Table 58: Stage 16N-R4F full-target native restart comparison results.

Case	Endpoint	Status	Max global error	Max primary local error	Diagnostic S11 error	Controlling
R4F1	500	fail	1.1887403%	11.83033%	0.92936729%	`HOLE_RING`
R4F2	750	review	0.31896796%	8.0369449%	1.5702038%	`HOLE_RING`

The R4F result is mixed. R4F2 improves strongly relative to R4E2/R4J8: the maximum primary local error falls from 13.598377% to 8.0369449%, and the diagnostic `HOLE_RING_S11_MAX_ABS` error falls from 13.011616% to 1.5702038%. This supports the hypothesis that material-only overwrite is incomplete on the 500–505–750 branch. R4F1, however, does not improve relative to R4E1/R4J7: the primary local error remains approximately 11.83%, controlled by `HOLE_RING_SDV8_MAX`. Because R4F1 uses a full native target restart at cycle 280 and no overwrite, this points away from material-only overwrite as the sole explanation.

The current decision is therefore to keep R4J9/R4J10 on hold. Material-only overwrite is a contributor, most clearly shown by the R4F2 improvement, but R4F1 remains evidence of an additional restart, comparison, extraction, or branch-specific issue. The next audit should focus on the 250–280–500 branch: native restart phase, reference/comparison mapping, selected-cycle local-state extraction, and the local maximum identity controlling `HOLE_RING_SDV8_MAX`.

19.16 R4G Native Restart and Source-Split Replay Controls Prepared

Because the R4F result is mixed, a two-day unattended R4G diagnostic batch was prepared before any R4J9/R4J10 extrapolated refinement. R4G separates direct native restart behavior from source-split replay behavior and checks both the 250-branch and 500-branch baselines.

Table 59: Stage 16N-R4G unattended native restart/replay controls prepared.

Case	Mode	Restart chain	First solved cycle	Endpoint
R4G1	direct original	250–500	251	500
R4G2	direct original	270–500	271	500
R4G3	direct original	280–500	281	500
R4G4	source split	250–270–500	271	500
R4G5	source split	250–280–500	281	500
R4G6	direct original	500–750	501	750

The original R1A restart deck explicitly requested restart writes at cycles 100, 250, and 500. Therefore, R4G2 and R4G3 also serve as practical availability checks for direct restart records at cycles 270 and 280 in the retained native restart files. If these cases fail at datacheck before solving, that should be classified as restart-record/setup evidence rather than a physical comparison failure. R4G4/R4G5 then test the corresponding source-split replay path.

Safety checks before submission showed no already-running PBS jobs for the user, `/home` at 12T used out of 17T, `/scratch` at 82T used out of 110T, and `/home/pr21vyci` at 930G. Three existing R4F `.stt` files larger than 10G were found in `/home`; they were left untouched because available capacity remained sufficient. The deployed R4G decks were checked before submission and the first solved steps matched the intended cycles.

Table 60: Stage 16N-R4G unattended submission queue.

Case	PBS job	Initial state	Initial node/comment
R4G1	1348008	running	mnode101
R4G3	1348009	running	mnode102
R4G2	1348010	queued	teachingq running-job limit
R4G6	1348011	queued	teachingq running-job limit
R4G4	1348012	queued	teachingq running-job limit
R4G5	1348013	queued	teachingq running-job limit

19.17 R4G Native Restart and Source-Split Replay Controls Completed

All six R4G controls completed with `Exit_status=0`. The direct original native-restart controls pass exactly, while the source-split replay controls reproduce the branch error.

Table 61: Stage 16N-R4G completed job resource history.

Case	PBS job	Host	Walltime	CPU time	CPU percent	Memory
R4G1	1348008	mnode101	03:44:18	20:33:11	651	94371840 kb
R4G2	1348010	mnode100	03:29:21	18:58:14	645	94375908 kb
R4G3	1348009	mnode102	03:16:19	18:04:31	644	94373888 kb
R4G4	1348012	mnode102	03:34:51	19:55:01	644	94377452 kb
R4G5	1348013	mnode101	03:34:57	20:06:26	653	94373888 kb
R4G6	1348011	mnode101	03:45:57	20:32:09	642	94377848 kb

Table 62: Stage 16N-R4G comparison classification.

Case	Mode	Status	Max global error	Max primary local error	Diagnostic S11 error	Co
R4G1	direct 250–500	pass	0%	0%	0%	nc
R4G2	direct 270–500	pass	0%	0%	0%	nc
R4G3	direct 280–500	pass	0%	0%	0%	nc
R4G4	split 250–270–500	review	2.5314199%	9.3860617%	1.2392968%	HO
R4G5	split 250–280–500	fail	1.1887403%	11.83033%	0.92936729%	HO
R4G6	direct 500–750	pass	0%	0%	0%	nc

This resolves the main ambiguity from R4F. Direct native restart and comparison mapping are clean: direct restarts from cycles 250, 270, 280, and 500 all pass exactly. The source-split replay path is the failure mechanism. R4G4 already drifts to review after source 250–270, and R4G5 reproduces the R4F1 failure after source 250–280 with the identical summary `500,fail,1.1887403,11.83033,0.92936729`. The controlling metric in both source-split cases is `HOLE_RING_SDV8_MAX`.

The current thesis conclusion should therefore not blame the direct native restart machinery or endpoint comparison procedure. The evidence points to source-split restart generation, source-split restart-file selection, or source-split phase history. R4J9/R4J10 remain on hold until the direct-original cycle-270/280 restart files are compared against the source-split cycle-270/280 restart files.

19.18 R4H Restart-Source Diagnostics Submitted

The R4G diagnostics show that direct native restart from the original reference is exact at cycles 250, 270, 280, and 500. However, source-split replay from 250 to 270 or 280 followed by a second restart introduces significant local hole-ring error. Therefore, the active issue is not endpoint comparison or direct Abaqus restart, but the generation or selection of source-split restart states. The next diagnostic stage, R4H, tests whether the inconsistency is caused by second-generation restart files, restart from the final step of short source jobs, or source-split phase/history effects.

A no-new-job audit was attempted on the retained R4G source jobs, but the source ODB extraction reported no selected field-output frames for the R4G4/R4G5 source jobs. The R4H source decks therefore explicitly request restart writes throughout the short source solve and field output at the interior restart target and source end cycles.

Table 63: Stage 16N-R4H restart-source diagnostics submitted.

Case	Mode	Restart source	First solved cycle	PBS job
R4H1	long replay	R4G1 cycle 280	`CYCLE_0281`	1349085
R4H2	interior source split	250–281, restart 280	`CYCLE_0281`	1349086
R4H3	long replay	R4G1 cycle 270	`CYCLE_0271`	1349087
R4H4	interior source split	250–271, restart 270	`CYCLE_0271`	1349088
R4H5	long replay	R4G6 cycle 505	`CYCLE_0506`	1349089
R4H6	interior source split	500–506, restart 505	`CYCLE_0506`	1349090

The safety gate before submission showed no pre-existing PBS jobs, `/home` at 14T used out of 17T with 2.9T available, `/scratch` at 82T used out of 110T with 29T available, and `/home/pr21vyci` at 3.6T. All six jobs use `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90gb`, `walltime=24:00:00`, and `teachingq`. R4H1 and R4H2 started immediately on mnode100 and mnode101 at 2026-06-17 16:55:59 CEST; R4H3–R4H6 remained queued behind the teachingq two-running-job limit. R4J9/R4J10 remain blocked until R4H is classified.

19.19 R4I Buffer Diagnostic Attempt Check

The R4I buffer diagnostic batch was checked on 2026-06-20. `qstat -u pr21vyci` returned no active jobs, so the four R4I jobs and the scratch archive inventory job were inspected through PBS history. All four R4I jobs reached `job_state=F` with `Exit_status=1` and `Stageout_status=1`, while the archive inventory job `1350236.mmaster02` finished with `Exit_status=0`. The R4I PBS failure is not an Abaqus solve failure: retained stdout shows that each source solve completed, each datacheck completed, each continuation solve completed, and the extraction step wrote cycle metrics plus selected-cycle loop/local-state CSVs.

Table 64: Stage 16N-R4I PBS history after the 2026-06-20 check.

Case	PBS job	Host	Status	Walltime	CPU time	Memory
R4I1	1350232	mnode101	post-solve workflow fail	05:13:15	20:07:50	94379784 kb
R4I2	1350233	mnode102	post-solve workflow fail	05:43:35	21:39:22	94377932 kb
R4I3	1350234	mnode101	post-solve workflow fail	05:56:36	22:10:57	94375908 kb
R4I4	1350235	mnode104	post-solve workflow fail	05:26:16	20:27:07	94375908 kb

The workflow failed at the comparison step because the expected reference CSV files were missing on the remote mirror. R4I1–R4I3 tried to open `../../../stage16n_1000cycle_pilot/stage16n_plat`

R4I4 tried to open `../../../stage16n_parallel_max_reference/stage16n_parallel_max_reference_1000c`
The post-solve exception caused the PBS failure state and prevented durable copy-back of the generated R4I comparison evidence.

The retained stdout still records the intended restart reads: R4I1 read source step 280 increment 54, R4I2 read source step 280 increment 56, R4I3 read source step 270 increment 60, and R4I4 read source step 505 increment 54. However, the generated R4I metrics and selected-cycle CSVs were not preserved in the `/home` mirror, and the follow-up inventory showed `/scratch/pr21vyci` at only 48K, so the scratch run directories were no longer available for recovery.

R4I therefore cannot yet be classified with the R4H/R4I decision table. The only safe conclusion is that the submitted solver jobs completed their Abaqus portions, but the diagnostic comparison evidence was lost due to a missing-reference postprocessing failure plus scratch cleanup. R4J9/R4J10 remain blocked until R4I is rerun or otherwise reconstructed with durable reference CSV access and durable lightweight copy-back.

19.20 R4I-R Recovery Rerun Prepared

The next action is an R4I recovery rerun, not R4J9/R4J10 and not scratch deletion. The cleanup inventory showed that the user scratch tree was already effectively empty: `/scratch/pr21vyci` used only 48K, the heavy-file inventory contained only the header row, and the delete-candidate list was empty. The cluster-wide `/scratch` usage therefore is not a current Stage 16N cleanup target.

R4I-R repeats the four R4I diagnostic branches with hardened postprocessing:

Table 65: Stage 16N-R4I-R recovery rerun cases.

Case	Source solve	Restart	Continuation	First solved cycle
R4I-R1	250–281 deck clone	280	281–500	`CYCLE_0281`
R4I-R2	250–300 buffered	280	281–500	`CYCLE_0281`
R4I-R3	250–300 buffered	270	271–500	`CYCLE_0271`
R4I-R4	500–525 buffered	505	506–750	`CYCLE_0506`

The generated recovery workflow uses a separate case root, `r4ir_restart_source_buffer_recovery`, and a separate dispatcher, `submit_stage16n_r4ir_restart_source_buffer_recovery_case.sh`. The PBS scripts follow the updated Stage 16N resource policy: queue `entryq`, `select=1:ncpus=16:mpiprocs=1` and `walltime=24:00:00`, with the maximum intended active usage still limited to two simultaneous 16-core jobs.

The important repair is evidence durability. Each PBS script copies the absolute reference CSVs into the scratch case directory before comparison. The runner then copies lightweight evidence back immediately after extraction, before any comparison command can abort. The copy-back includes CSV, markdown, text, status, log, and `.sta` files while excluding heavy Abaqus artifacts such as `.odb`, `.stt`, `.res`, `.sim`, `.mdl`, `.prt`, `.dat`, and `.msg`. The rerun also sets `KEEP_SCRATCH=1`; scratch directories should be deleted only after comparison summaries are copied, classified, and committed.

The recovery files were committed and pulled into the HPC scratch clone before submission. The storage gate passed on 2026-06-20 at 06:55 CEST, reporting `/home/pr21vyci` at 66G and no large Abaqus files in the home project. Early setup attempts `1350597/1350598` and `1350599/1350600` finished immediately with `Exit_status=23` because the PBS source directory was derived incorrectly. Commit `8a18cd5` fixed this by making each PBS script walk upward to the Git checkout root. The first two recovery cases then started successfully through the guarded Abaqus wrapper on 2026-06-20 at 07:01 CEST: R4I-R1 as `1350601.mmaster02` on mnode013 and R4I-R2 as 1350602.mmaster02 on mnode014. Both requested one node with 16 cores, 90 GB, and 24 h.

The first R4I-R wave is classified. R4I-R1, the deck-clone/truncated clean source at restart 280, passed with zero global, primary-local, and diagnostic S11 error at cycle 500. R4I-R2, the generated buffered source for the same restart target, failed/reviewed with a dominant HOLE_RING_SDV8_MAX mismatch: 1.1887403% maximum global error, 11.83033% maximum primary-local error, and 0.92936729% diagnostic S11 error at cycle 500. This separated the deck shape from the buffer length: the deck clone repairs the branch, while a longer generated source does not.

The second R4I-R wave was submitted after a new guarded storage gate on 2026-06-21 at 05:59 CEST. The gate reported /home 17T total, 13T used, 3.4T free; /scratch 110T total, 82T used, 28T free; /home/pr21vyci 66G; project 56G; and no large Abaqus files in the home project. Both second-wave jobs completed Abaqus and comparison with Exit_status=0. The results are:

Table 66: Stage 16N-R4I-R recovery rerun comparison results.

Case	PBS job	Source style	Cycle	Global error	Primary-local error	S11 error
R4I-R1	1350601	deck clone	500	0.0000000%	0.0000000%	0.0000000%
R4I-R2	1350602	buffered generated	500	1.1887403%	11.83033%	0.92936729%
R4I-R3	1350673	buffered generated	500	2.5314199%	9.3860617%	1.2392968%
R4I-R4	1350674	buffered generated	750	0.31896796%	8.0369449%	1.5702038%

The second-wave PBS accounting also shows limited average CPU utilisation over the whole restart-plus-postprocess workflow:

Table 67: Stage 16N-R4I-R second-wave PBS resource accounting.

Case	PBS job	Host	Walltime	CPU time	Average cores
R4I-R3	1350673	mnode001	06:56:02	27:40:33	3.99 / 16
R4I-R4	1350674	mnode002	05:20:51	19:52:33	3.72 / 16

Although both R4I-R3 and R4I-R4 were correctly launched with 16 Abaqus threads, PBS accounting showed weak average utilisation over the full workflow. R4I-R3 used 27:40:33 CPU time over 06:56:02 walltime, corresponding to approximately 3.99 average active cores out of 16, or 24.9% efficiency. R4I-R4 used 19:52:33 CPU time over 05:20:51 walltime, corresponding to approximately 3.72 average active cores out of 16, or 23.2% efficiency. This likely reflects serial and I/O-dominated phases such as restart reading, ODB writing, output extraction, copy-back, and comparison/postprocessing, rather than an incorrect Abaqus launch configuration. R4I-R5 and R4I-R6 should therefore keep the same 16-core, 90 GB, 24 h setup until they are classified, so the deck-clone diagnostic does not introduce a new resource variable.

The R4I-R decision is now stronger than the original R4I attempt because lightweight evidence was preserved and committed. R4I-R1 shows that cloned/truncated clean replay can make restart 280 exact. R4I-R2, R4I-R3, and R4I-R4 show that generated buffered source decks remain non-equivalent at restart 280, restart 270, and restart 505, even when the source solve extends beyond the restart target. Therefore the generated source-split restart path should not be used as a cycle-jump foundation.

The R4I-R recovery diagnostics show that the source-split error is not caused by insufficient post-target buffering. Generated buffered source decks at target cycles 270, 280, and 505 still reproduce the previously observed review/failure signatures. In contrast, the deck-clone/truncate source case at target cycle 280 passes exactly. Therefore, the restart inconsistency is attributed to the generated short-source deck construction rather than Abaqus native restart, endpoint comparison, or lack of post-target source buffer. The workflow direction is now to retire generated short-source decks unless their exact construction difference is found and corrected, and to treat deck-clone/truncate as the candidate fix.

R4I-R5 and R4I-R6 were submitted as the next two-job deck-clone confirmation wave on 2026-06-21 after the guarded storage gate passed. The gate reported `/home` 17T total, 13T used, 3.4T free; `/scratch` 110T total, 82T used, 28T free; `/home/pr21vyci` 66G; project 56G; and no large Abaqus files in the home project. Both jobs were submitted through the guarded R4I-R dispatcher, not raw unguarded `qsub`. Both requested `select=1:ncpus=16:mpiprocs=1:ompthreads=16:mem=90g` `walltime=24:00:00`, and executed in scratch under `/scratch/pr21vyci/stage16n_r4ir/`.

R4I-R5, `1350742.mmaster02`, deck-clone source 250–271 with restart 270–500, completed with `Exit_status=0`. The comparison at cycle 500 passed with zero maximum global error, zero maximum primary-local error, and zero diagnostic S11 error. This confirms that deck-clone/truncate repairs the 270–500 branch in the same way that R4I-R1 repaired the 280–500 branch.

R4I-R6, `1350743.mmaster02`, deck-clone source 500–506 with restart 505–750, did not reach a scientific comparison. The source solve and source extraction completed, and the continuation datacheck passed, but the continuation Abaqus solve failed with `WriteAll: Input/output error` while writing the continuation `.stt` file on scratch. The `.sta` tail ended at cycle 731 and reports `THE ANALYSIS HAS NOT BEEN COMPLETED`. PBS recorded `Exit_status=1` and `Stageout_status=0`. This is classified as an infrastructure/storage I/O failure, not as evidence that deck-clone/truncate fails for the 505–750 branch.

Case	PBS job	Host	Walltime	CPU time	Average cores
R4I-R5	1350742	mnode043	05:19:55	21:06:33	3.96 / 16
R4I-R6	1350743	mnode051	05:36:52	23:30:31	4.19 / 16

Table 68: Stage 16N-R4I-R deck-clone confirmation PBS resource accounting. R4I-R6 failed by scratch I/O before comparison.

The deck-clone diagnostic gate is therefore partially advanced but not complete. R4I-R5 passes and confirms the 270–500 branch. R4I-R6 must be rerun or otherwise recovered after addressing the scratch `.stt` I/O/quota condition before the 505–750 deck-clone branch can be classified. Future short-source generation should still be replaced by deck-clone/truncate only after the remaining 505–750 branch passes, followed by deck-clone exact controls R4K1 and R4K2 before any true extrapolated jump cases are unblocked. R4J9/R4J10 remain blocked.

R4I-R6B was prepared as a single recovery job for the 505–750 branch. The source deck remained restart-writing because it must create the cycle-505 restart record, while the continuation deck was made restart-read-only with no `*Restart, write` request. Before submission, the scratch quota diagnosis was corrected: `/scratch/pr21vyci` is a symlink to `/scratch9/pr21vyci`, so `du -sh /scratch/pr21vyci` only measured the symlink. The true scratch target still contained the failed R4I-R6 scratch directory. After the lightweight R4I-R6 evidence had already been preserved, the failed R4I-R6 scratch run directory was removed, freeing about 508 GB including the failed 438 GB continuation `.stt`. The R4I-R6B guarded storage gate then passed and the job was submitted as `1350763.mmaster02`.

R4I-R6B did not reach the intended continuation or comparison. PBS recorded `Exit_status=1`, `Stageout_status=1`, walltime 00:05:21, and CPU time 00:11:04. The Abaqus source solve failed with `WriteAll: Input/output error` while writing the source `.stt` file on scratch. The source `.sta` tail reached `CYCLE_0503` and then ended with `THE ANALYSIS HAS NOT BEEN COMPLETED`. The failed scratch run directory was about 2.6 GB at inspection, dominated by a partial 2.23 GB source `.stt`. This is classified as *source restart-write infrastructure failure before scientific comparison*. It does not show that deck-clone/truncate fails for the 505–750 branch, and it also does not test the no-continuation-restart-write hypothesis because the job stopped before the continuation solve.

Case	PBS job	Host	Status	Walltime	CPU time
R4I-R6B	1350763	mnode013	source `.stt` I/O failure	00:05:21	00:11:04

Table 69: Stage 16N-R4I-R6B recovery accounting. The job failed during the source restart-write stage before continuation or comparison.

Later R4K, R4I-R6 recovery, or resource-calibration jobs should add explicit phase timing around the source solve, continuation solve, ODB extraction, copy-back, and comparison. The timing wrapper should print a phase label, start/end timestamps, exit code, and `/usr/bin/time -v` resource report for each major command. Because R4I-R5 and the failed R4I-R6 workflow again averaged roughly four active cores over the full allocation, the next resource study should compare one representative case at 8 cores versus 16 cores, holding the numerical setup fixed and comparing walltime, CPU efficiency, and numerical results before changing the default Stage 16N production allocation. However, the next scientific gate remains one clean 505–750 deck-clone completion and comparison; R4K and R4J9/R4J10 remain blocked until that evidence exists.

The uploaded lightweight evidence for R4I-R3 through R4I-R6B is in `runs/chaboche_umat/stage16_abaqus` Completed comparison case directories contain the case status markdown, comparison summary/details CSVs, cycle metrics CSVs, selected-cycle loop/local-state CSVs, copied reference CSVs, PBS stdout, and `qstat_<jobid>_finished_full.txt`. The R4I-R6 directory contains the source extraction CSVs, small Abaqus logs, `qstat_1350743_finished_full.txt`, and a continuation `.sta` tail documenting the scratch I/O failure. The R4I-R6B directory contains the status markdown, PBS stdout, source log, source `.sta` tail, Abaqus exception file, and `qstat_1350763_finished_full.txt` documenting the source restart-write I/O failure.

A Key Repository Evidence Files

- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_parallel_ma`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_adaptive_de`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_adaptive_de`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_REINJECTION`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_FIXED_JUMP_`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_make_b1_rob`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_STATEV_INDE`
- `runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_audit_umat_`

- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_NATIVE_REST
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/prepare_stage16n_res
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/run_stage16n_native_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/prepare_stage16n_r3_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_compare_nat
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/prepare_stage16n_r4h
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/submit_stage16n_r4h_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con

- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- hpc_lightweight_stage_archive/
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/prepare_stage16n_r4g
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/submit_stage16n_r4g_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/prepare_stage16n_r4f
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/submit_stage16n_r4f_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_restart_con
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_PROFESSOR_U
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_

- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_fixed_jump_
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/stage16n_exact_reinj
- runs/chaboche_umat/stage16_abaqus_inhomogeneous_cycle_jump_benchmark/STAGE16N_HPC_MAIL_PO